

From Zero to Hero

cisco Live !

How to successfully implement Data Center Automation using Service as Code

Peter Charpentier
Principal Architect, Cisco CX

Morten Skriver
Principal Architect, Cisco CX

Webex App

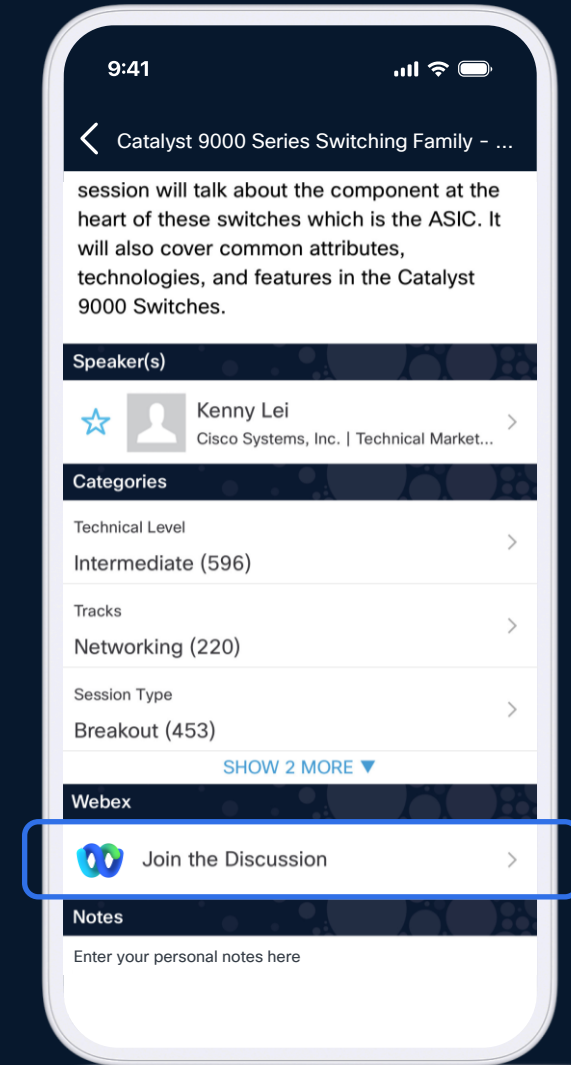
Questions?

Use Webex App to chat with the speaker after the session

How

- 1 Find this session in the Cisco Events App
- 2 Click “Join the Discussion”
- 3 Install the Webex App or go directly to the Webex space
- 4 Enter messages/questions in the Webex space

Webex spaces will be moderated by the speaker until February 27, 2026.



Agenda

Demo

Automation Drivers &
Services as Code

Organizational Challenges

ACI as Code & Firepower as Code

Standardize IT offerings &
Conclusions



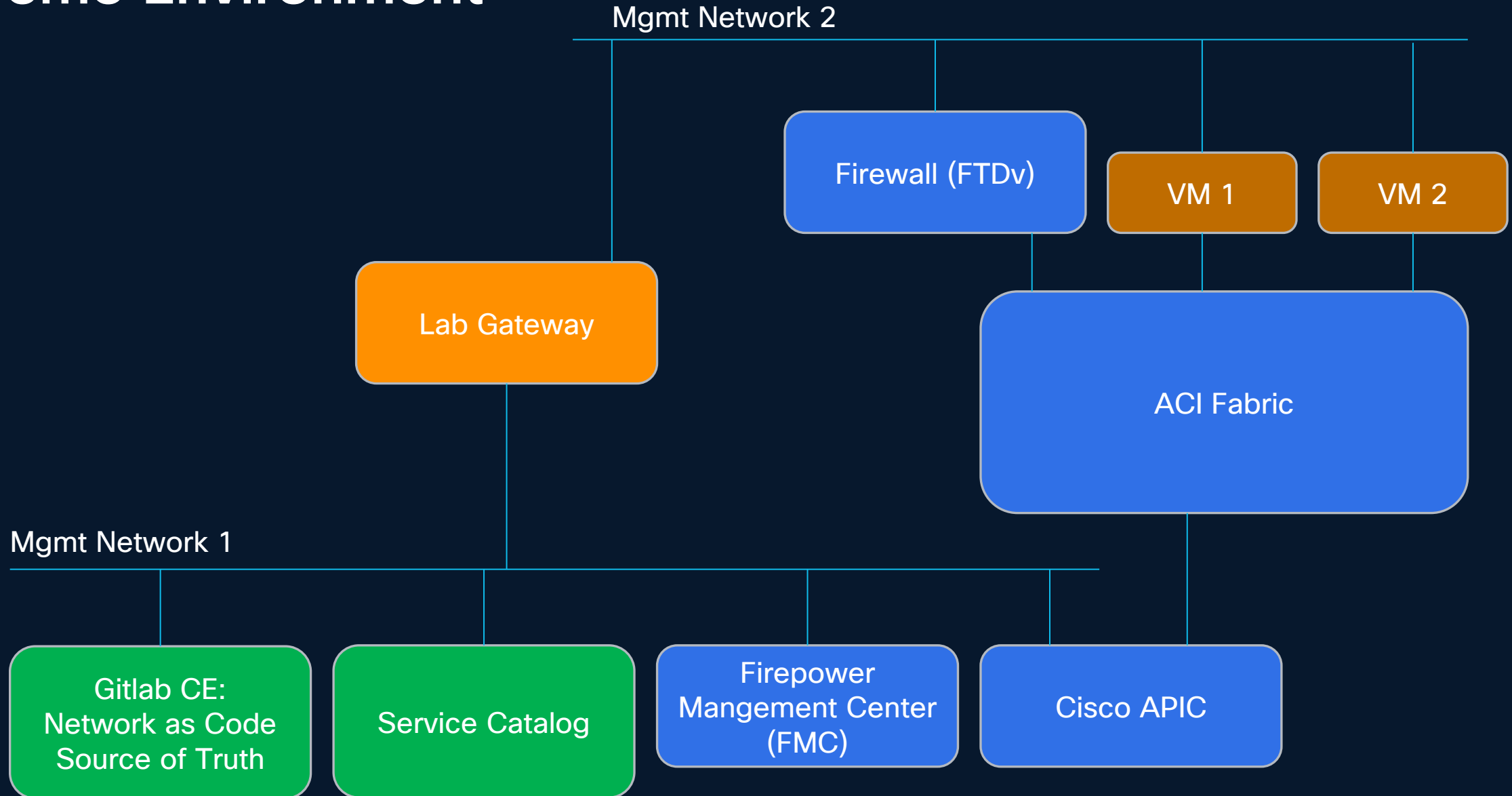


**Let's start off with a demo of the
end result....**

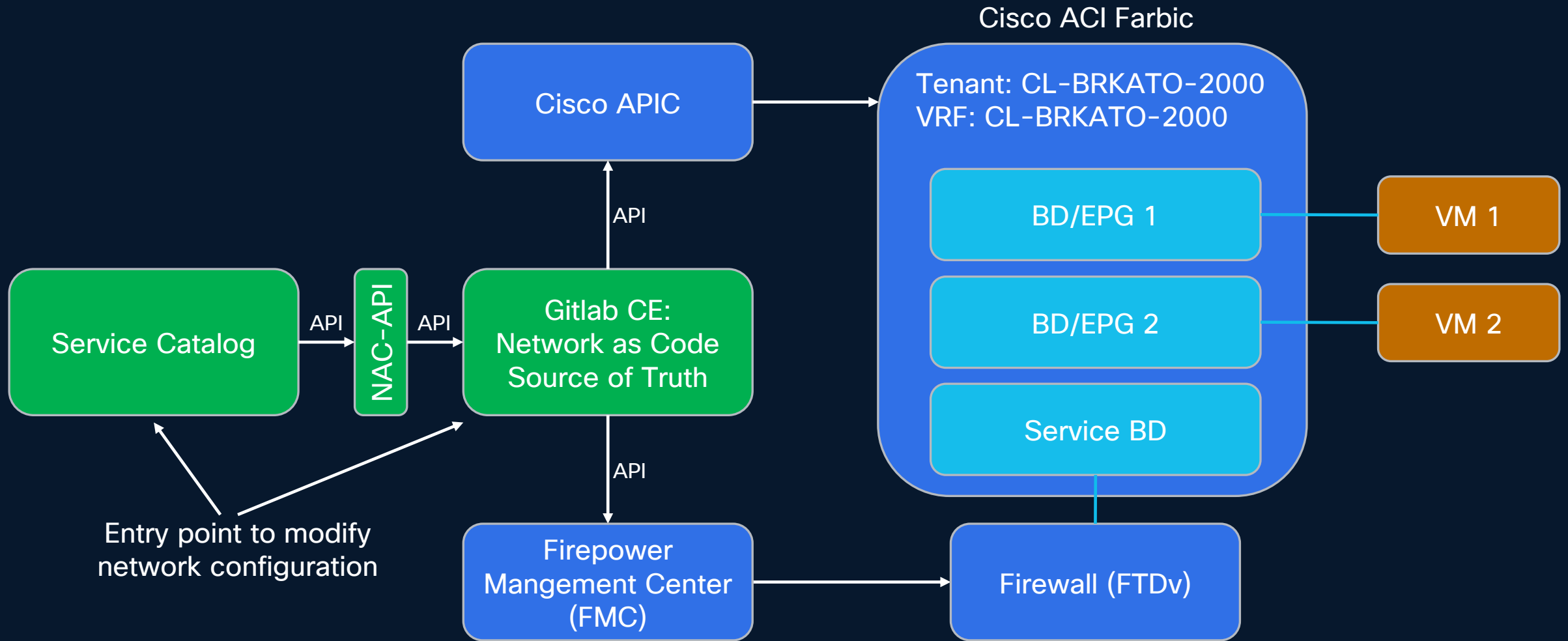
The background of the slide is a dark blue field filled with dynamic, flowing light streaks. These streaks are primarily in shades of bright blue and cyan, with some warmer orange and pinkish tones visible in the upper right quadrant. The lines are curved and layered, creating a sense of depth and movement, reminiscent of light trails or data flow.

Demo

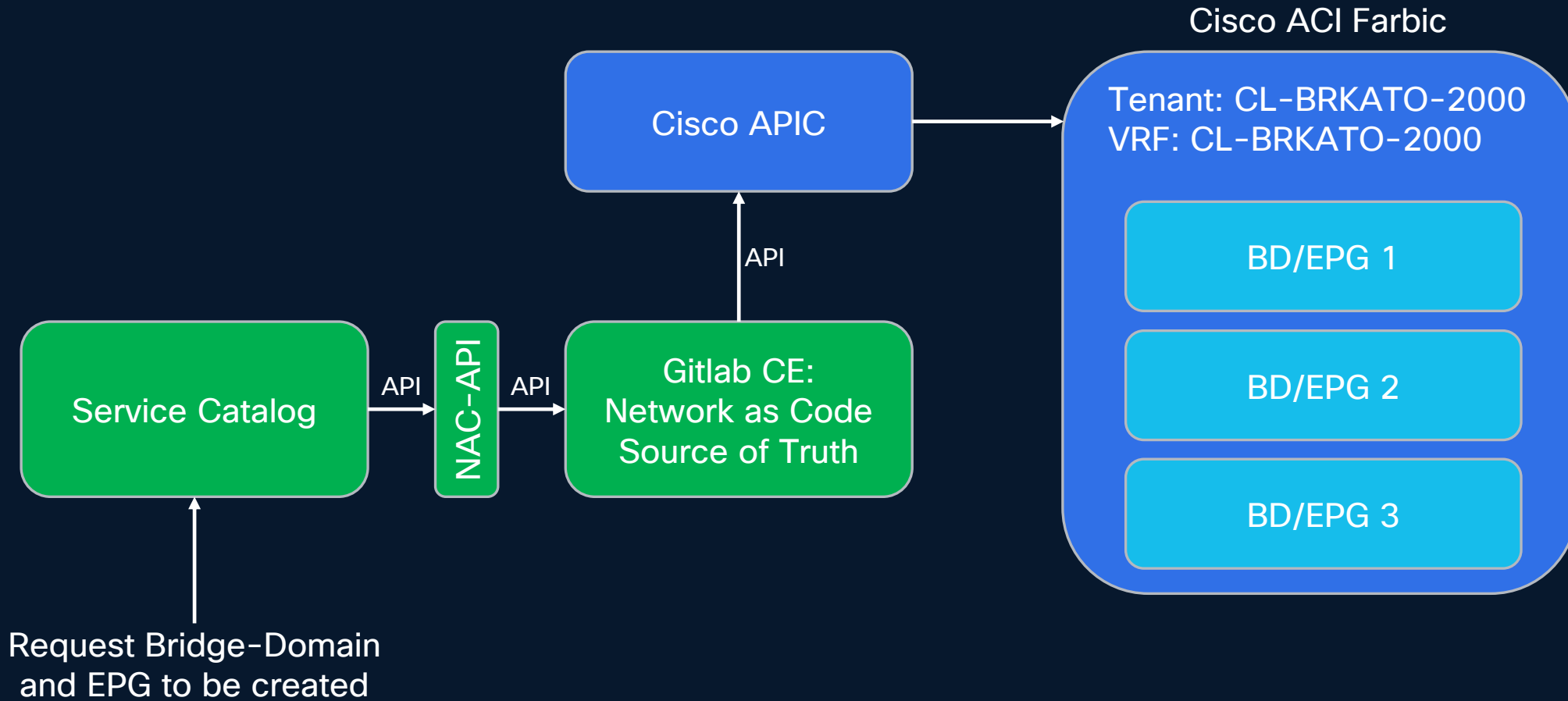
Demo Environment



Demo Environment

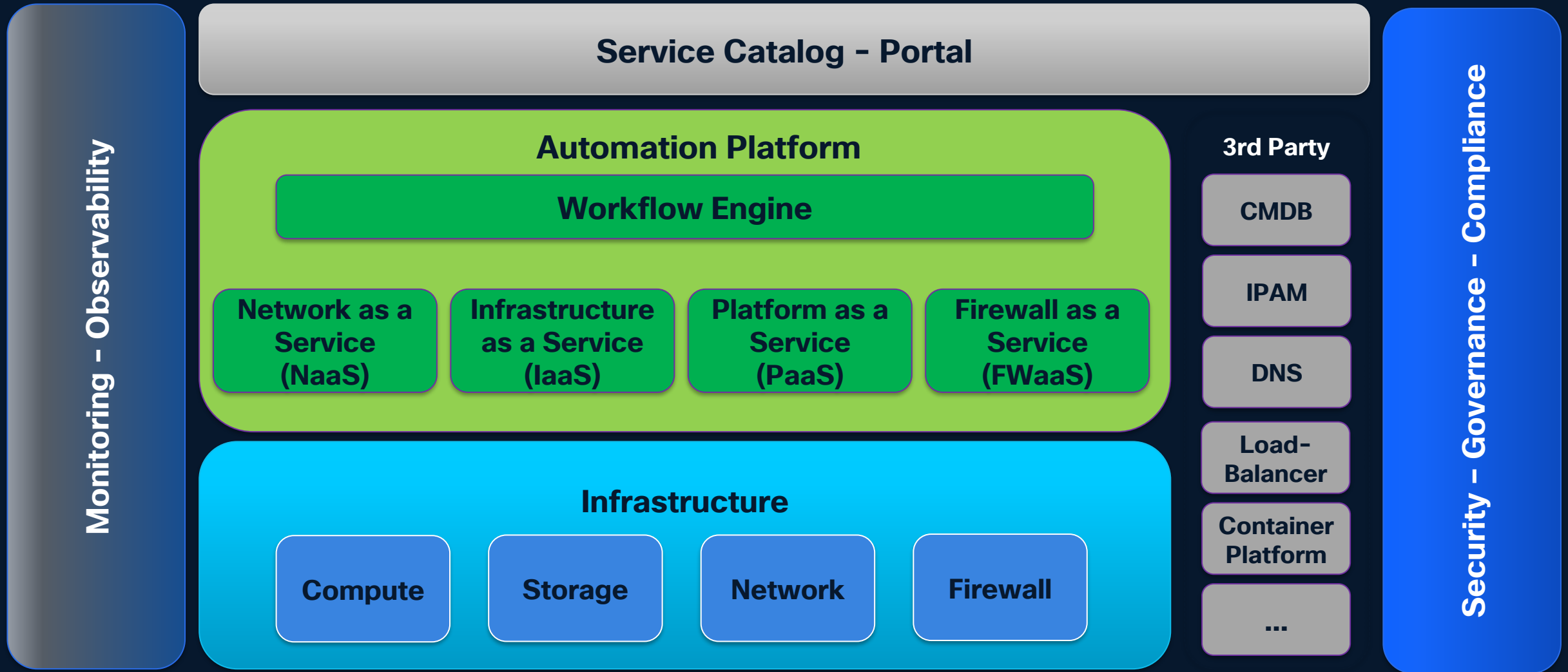


Demo – Deploy ACI Bridge Domain



Automation Drivers

Automation Architecture Overview



Business Drivers for Automation

Time to Market

Infrastructure Cost

Availability / Uptime

Ease of Consumption

Business Drivers for Automation



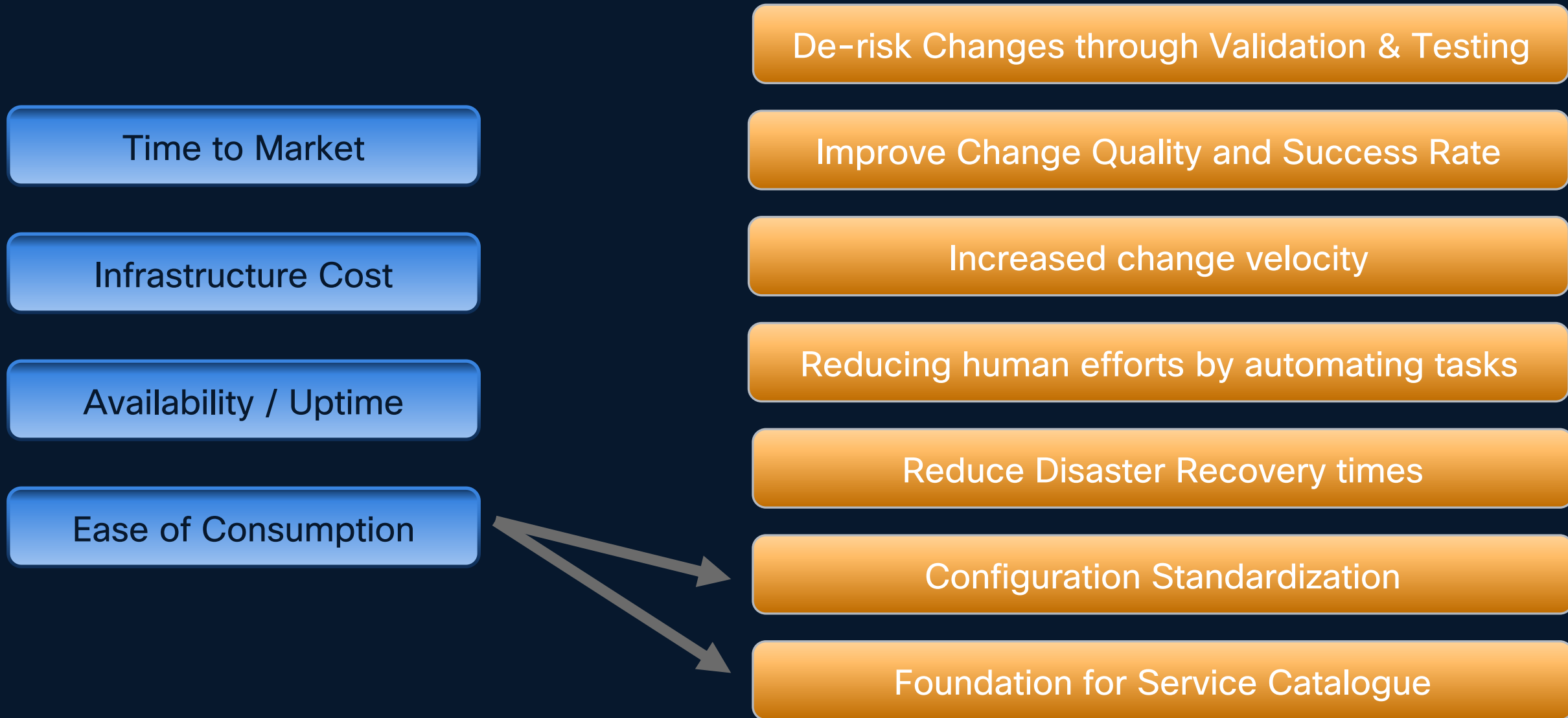
Business Drivers for Automation



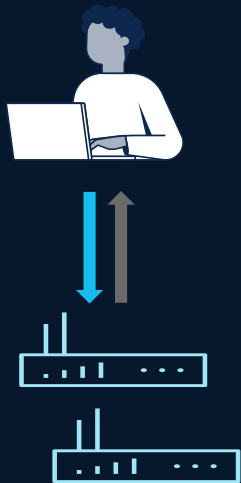
Business Drivers for Automation



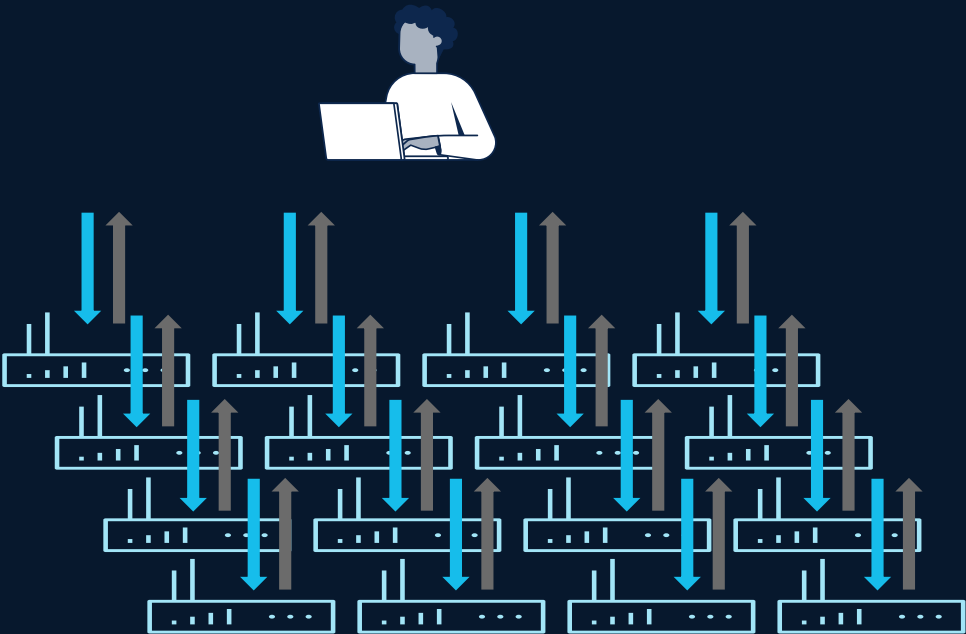
Business Drivers for Automation



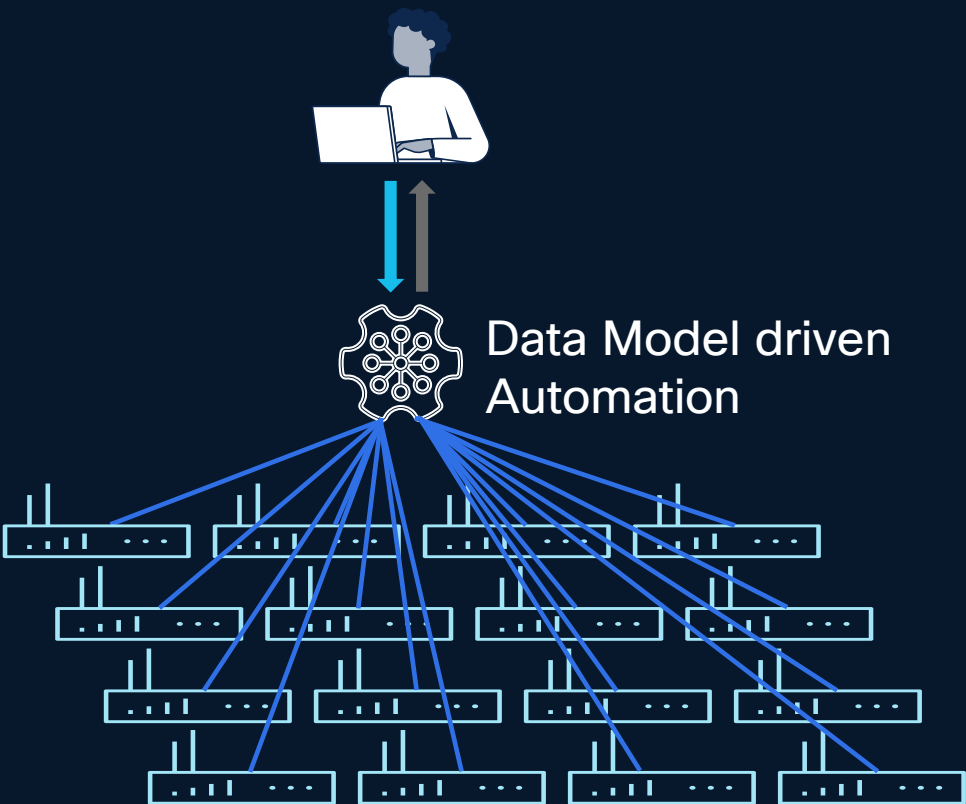
Approaches for Managing Configuration



Manually - weeks

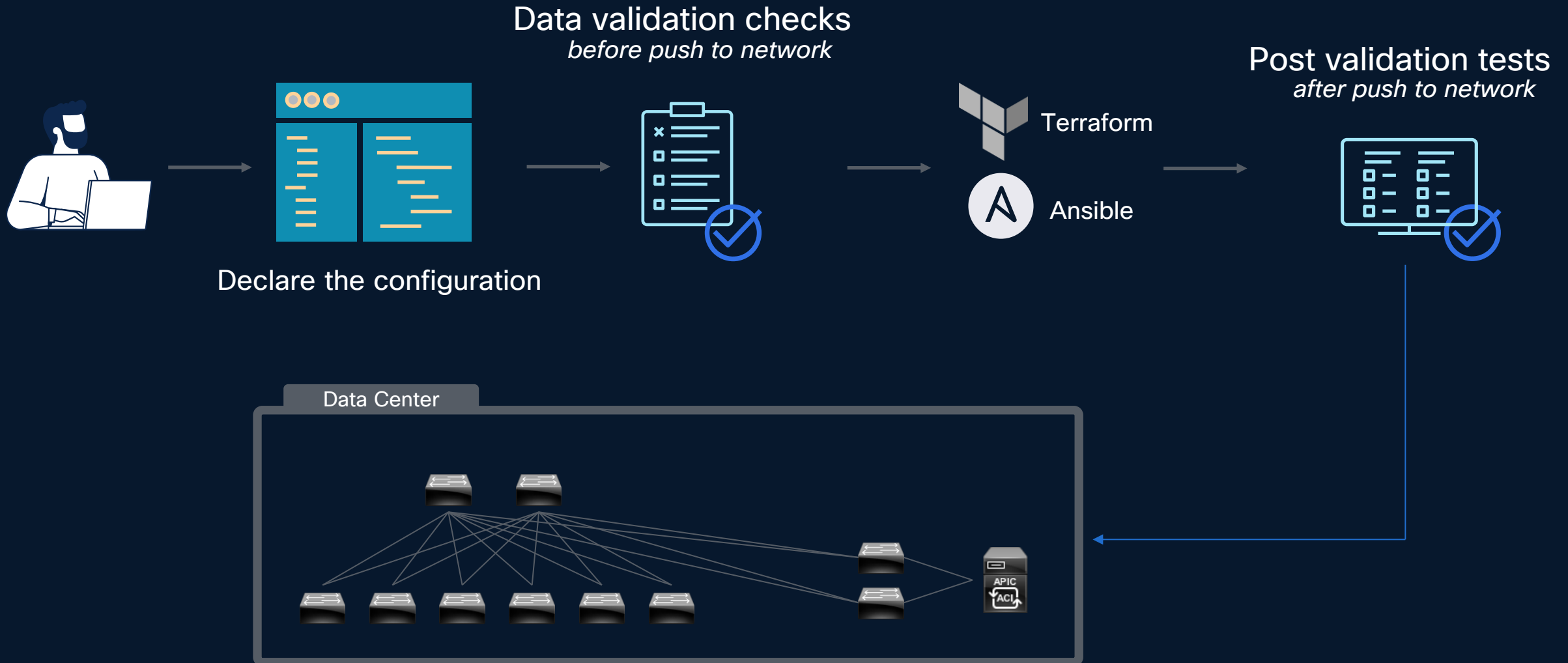


Scripting - days



Intent Based - hours

Basics of Infrastructure as Code



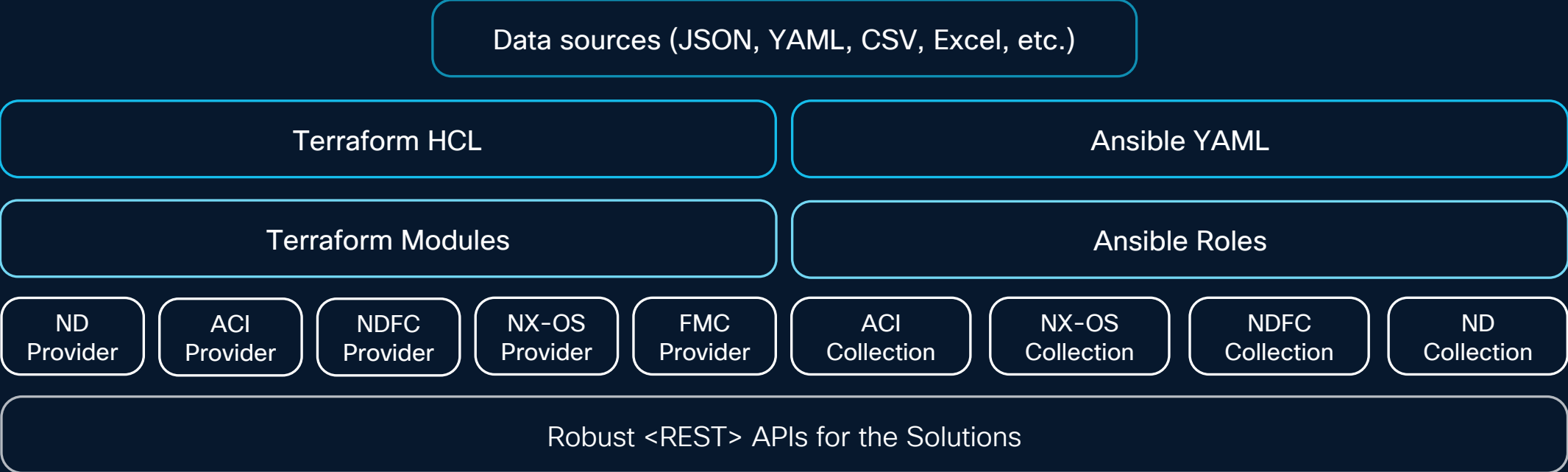
Services as Code

Programmability Strategy

Abstraction

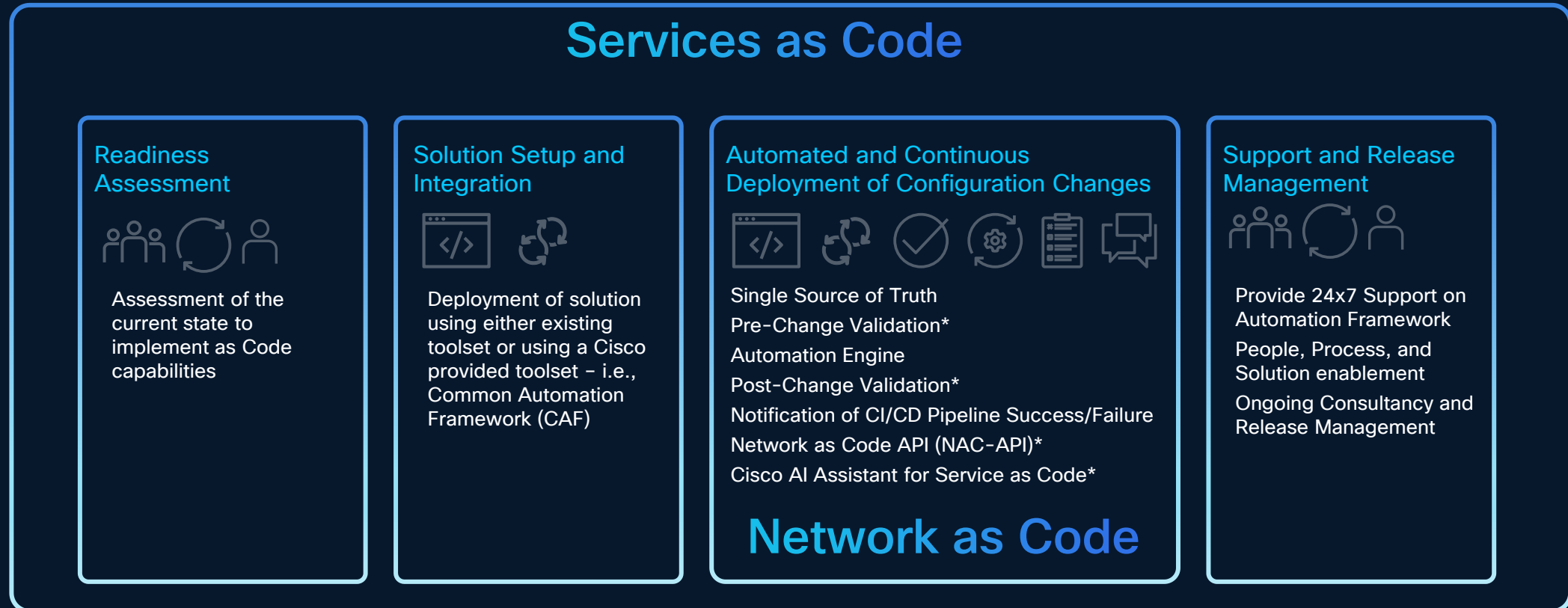


Flexibility



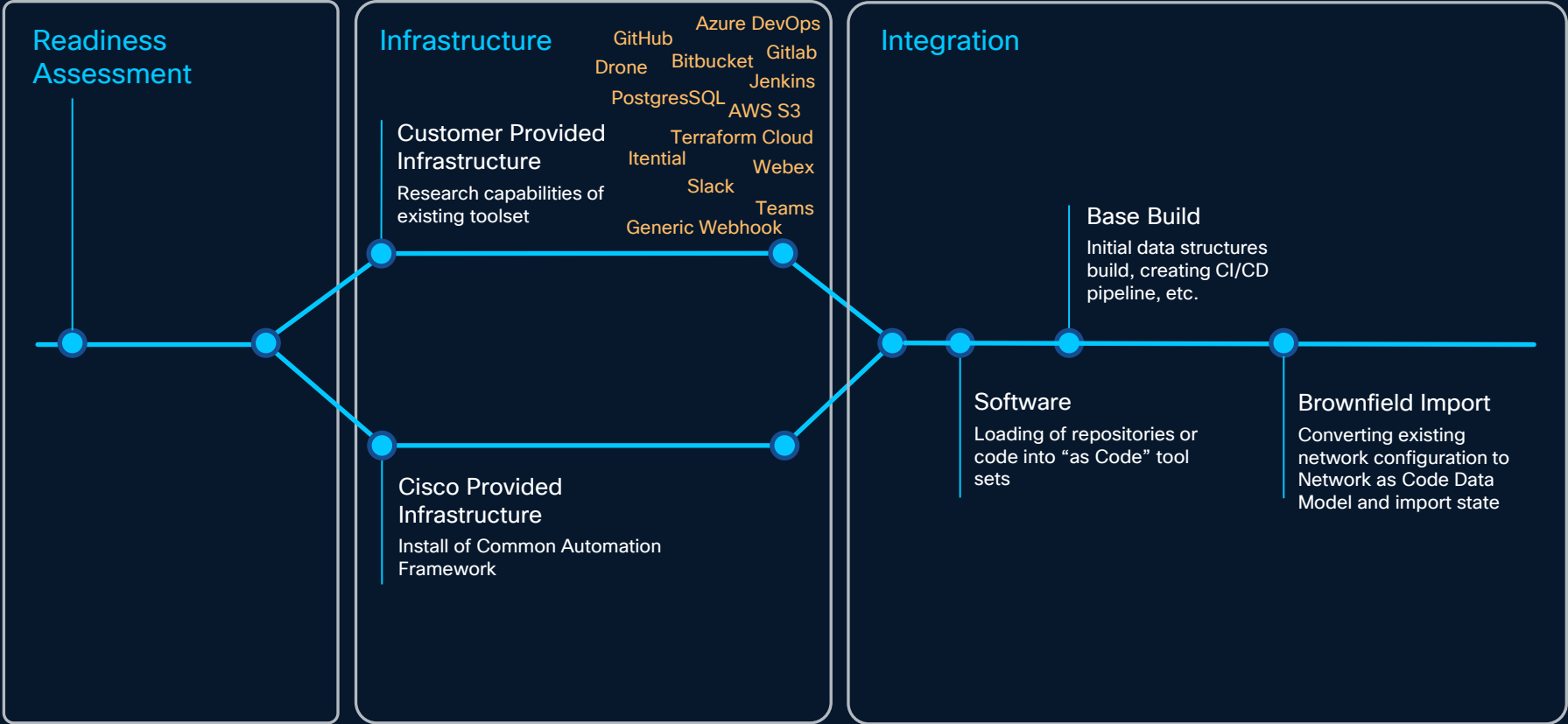
Our ambition is to empower each type of user

Services as Code – High Level View

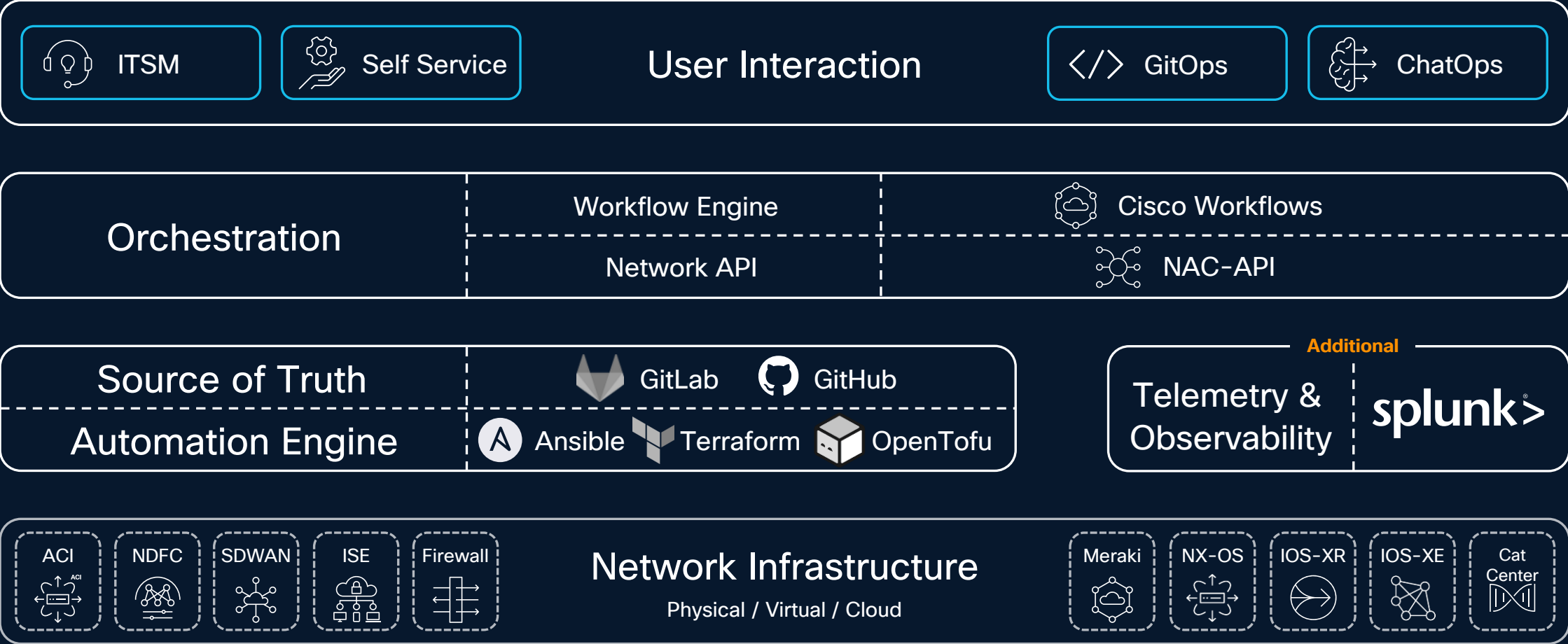


* Network as Code functionality available through an active Cisco CX Service Engagement

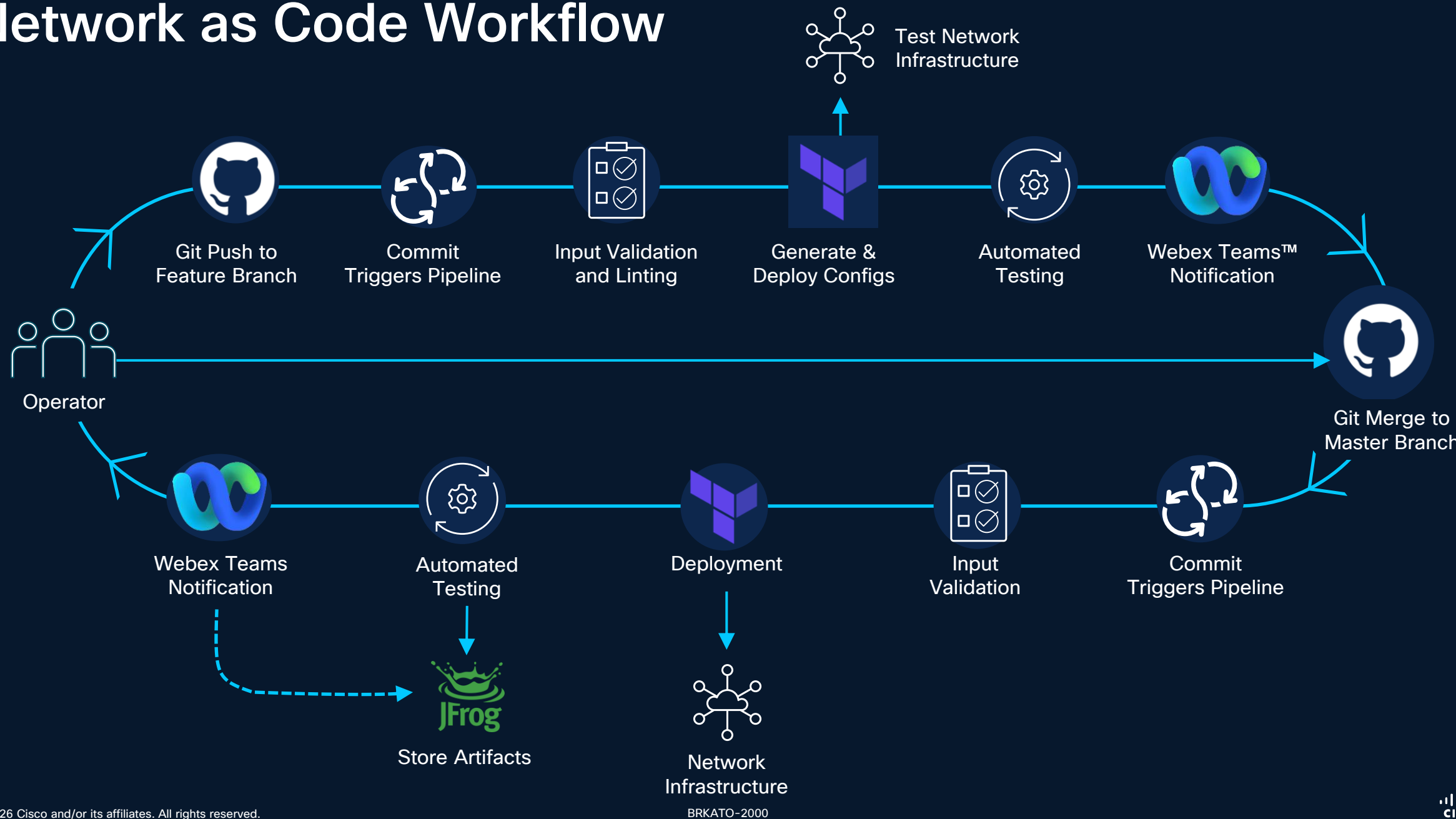
Services as Code - Solution Setup and Integration



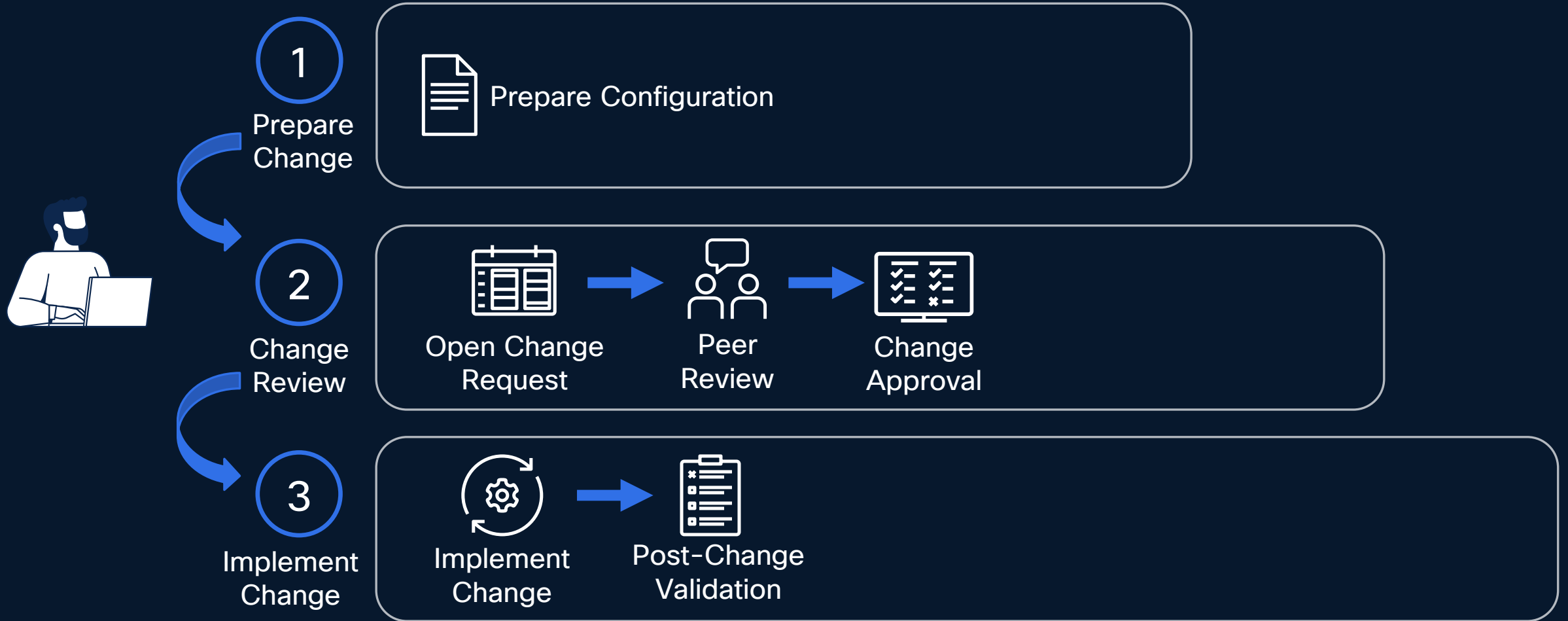
Network as Code Architecture



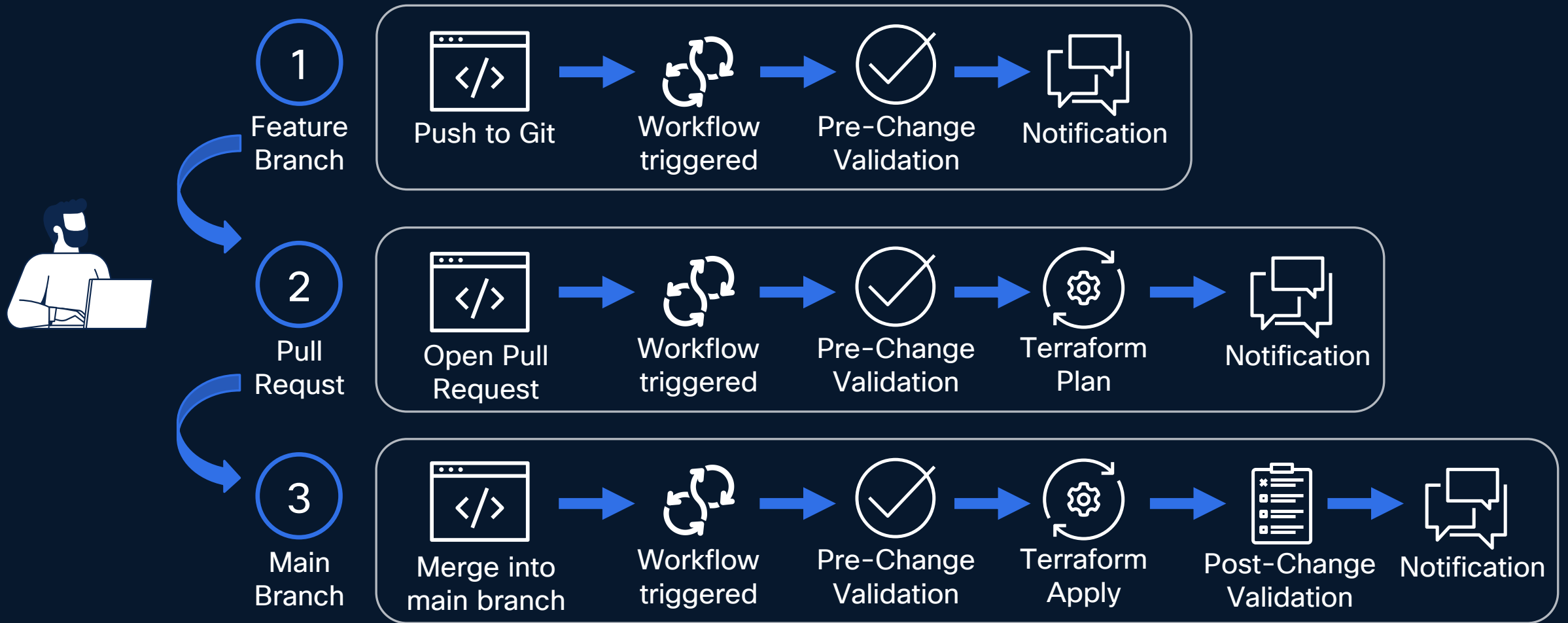
Network as Code Workflow



Traditional Change Workflow



Network as Code Workflow

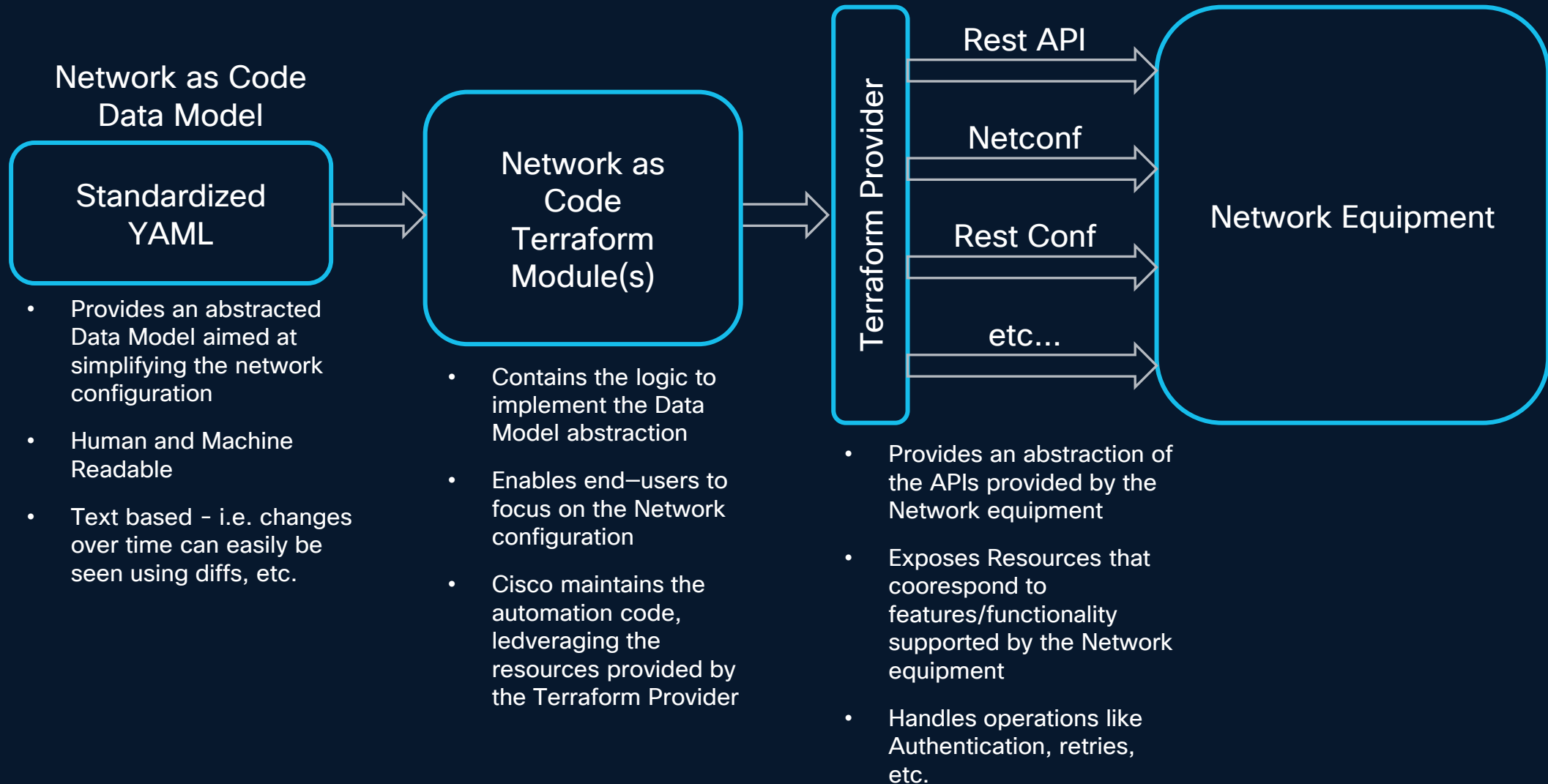


Terraform/OpenTofu Primer

- Terraform is a tool developed by HashiCorp for provisioning and managing infrastructure using a **declarative** configuration language
- OpenTofu is the open-source equivalent of Terraform – Network as Code supports both
- Compose and combine infrastructure resources to build and maintain **desired state**
- Manages all resources through **APIs**
- **Maintains state** for the resources managed/deployed by Terraform
- Terraform uses **core and plugin** components for basic functions and extensibility
- **Desired state/configuration** specified using HashiCorp Configuration Language (HCL)

```
provider "aci" {  
    username = "admin"  
    password = "cisco123"  
    url      = "https://10.1.1.1"  
}  
  
resource "aci_tenant" "example_tenant" {  
    name = "ciscolive"  
}  
  
resource "aci_vrf" "example_vrf" {  
    tenant_dn = aci_tenant.example_tenant.id  
    name      = "vrf1"  
}
```

Terraform/OpenTofu Primer – Providers and Modules



Simplification through Data Model driven Automation

Product Configuration Model
API - native

```
{
  "totalCount": "1",
  "imdata": [
    {
      "fvTenant": {
        "attributes": {
          "annotation": "",
          "descr": "",
          "dn": "uni/tn-CX",
          "name": "CX",
          "nameAlias": "",
          "ownerKey": "",
          "ownerTag": "",
          "userdom": ":all:"
        },
        "children": [
          {
            "vnsSvcCont": {
              "attributes": {
                "annotation": "",
                "userdom": ":all:"
              }
            }
          }
        ]
      }
    }
  ],
}
```

```
{
  "fvRsTenantMonPol": {
    "attributes": {
      "annotation": "",
      "tnMonEPGPName": "",
      "userdom": ":all:"
    }
  },
  "fvEpTags": {
    "attributes": {
      "annotation": "",
      "userdom": ":all:"
    }
  },
  "fvCtx": {
    "attributes": {
      "annotation": "",
      "bdEnforcedEnable": "no",
      "descr": "",
      "ipDataPlaneLearning": "enabled",
      "knwMcastAct": "permit",
      "name": "VRF1",
      "nameAlias": "",

```

...

+200 Lines of Configuration

Simplification through Data Model driven Automation

Infrastructure as Code Model

Terraform Provider - native

```
resource "aci_tenant" "tenant_CX" {
  name = "CX"
}

variable "vrfs" {
  default = {
    VRF1 = {
      name = "VRF1"
    },
    VRF2 = {
      name = "VRF2"
    }
  }
}

resource "aci_vrf" "vrfs" {
  for_each = var.vrfs
  tenant_dn = aci_tenant.tenant_CX.id
  name      = each.value.name
}
```

20 Lines of Configuration

Simplification through Data Model driven Automation

Network as Code Data Model

```
apic:
  tenants:
    - name: CX
      vrfs:
        - name: VRF1
        - name: VRF2
```

6 Lines of Configuration

Simplification through Data Model driven Automation

Product Configuration Model API - native

```
{
  "totalCount": "1",
  "imdata": [
    {
      "fvTenant": {
        "attributes": {
          "annotation": "",
          "descr": "",
          "dn": "uni/tn-CX",
          "name": "CX",
          "nameAlias": "",
          "ownerKey": "",
          "ownerTag": "",
          "userdom": ":all:"
        },
        "children": [
          {
            "vnsSvcCont": {
              "attributes": {
                "annotation": "",
                "userdom": ":all:"
              }
            },
            {
              "fvRsTenantMonPol": {
                "attributes": {
                  "annotation": "",
                  "tnMonEPGPName": "",
                  "userdom": ":all:"
                }
              },
              {
                "fvEpTags": {
                  "attributes": {
                    "annotation": "",
                    "userdom": ":all:"
                  }
                },
                {
                  "fvCtx": {
                    "attributes": {
                      "annotation": "",
                      "bdEnforcedEnable": "no",
                      "descr": "",
                      "ipDataPlaneLearning": "enabled",
                      "knwMcastAct": "permit",
                      "name": "VRF2",
                      "nameAlias": ""
                    }
                  }
                }
              }
            ]
          }
        ]
      }
    }
  ]
}
```

+200 Lines of Configuration



TF-
Provider

Infrastructure as Code Model TF provider - native

```
resource "aci_tenant" "tenant_CX" {
  name = "CAB"
}

variable "vrfs" {
  default = {
    VRF1 = {
      name = "VRF1"
    },
    VRF2 = {
      name = "VRF2"
    }
  }
}

resource "aci_vrf" "vrfs" {
  for_each = var.vrfs
  tenant_dn = aci_tenant.tenant_CX.id
  name      = each.value.name
}
```

20 Lines of Configuration



TF-
Module

Network as Code Data Model

```
apic:
  tenants:
    - name: CX
      vrfs:
        - name: VRF1
        - name: VRF2
```

6 Lines of Data

Benefits of Network as Code

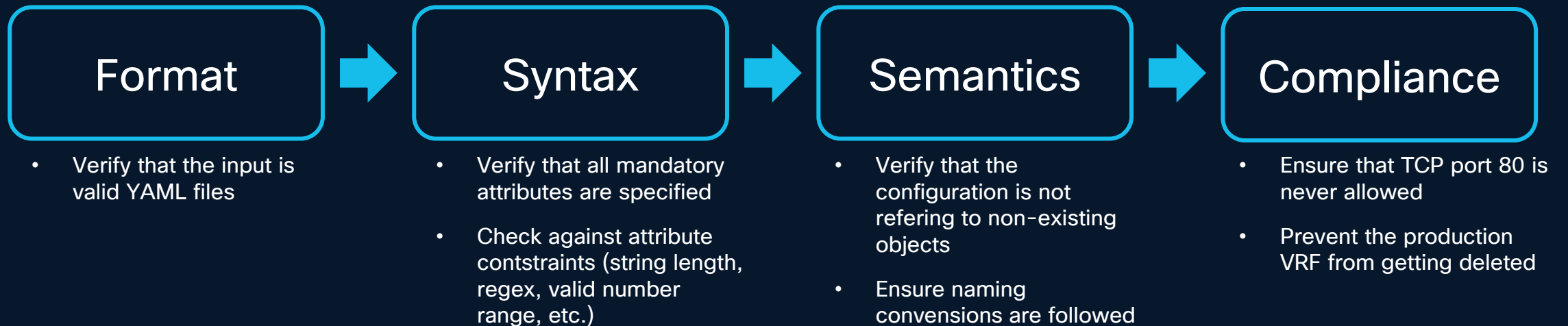
- Same configuration look & feel across architectures
- Highly simplified and abstracted
- Best Practices baked in
- Toolset independent “Open Source”

Pre-Change Validation

As the complexity of the configuration and the underlying data model increases, automated validation before deploying anything in a production environment becomes a critical aspect.

Several checks can be performed to ensure that the provided input data is not only valid, but also conform to common best practices and compliance rules.

Can to some extent be thought of as the equivalent of unit/integration testing in the software development world



Post-Change Testing

Some aspects can only be verified after deployment to determine whether a change was successful, making it essential to check device state and overall service health to ensure the network behaves as intended.

Post-deployment testing confirms that the actual network state matches the intended design and that critical configurations are properly applied.

These validations typically fall into three categories:

Configuration Tests

Verify if the desired configuration is present

Health Tests

Leverages build-in platform fault correlation (if present) to retrieve fault and health scores

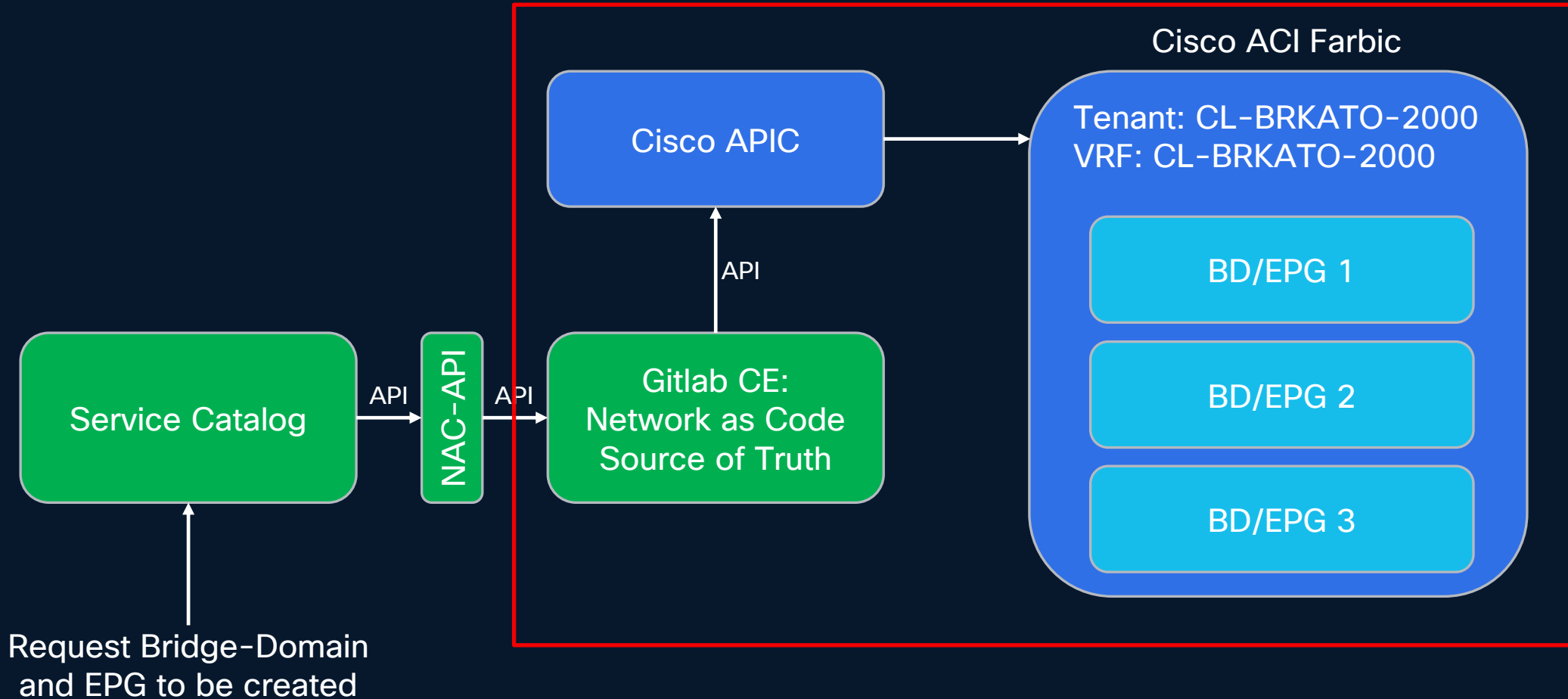
Operational Tests

Verify operational state according to input data, e.g. BGP peering state, etc.

The background of the slide is a dark blue field filled with dynamic, flowing light streaks. These streaks are primarily in shades of bright blue and cyan, with some warmer orange and pinkish tones visible in the upper right quadrant. The lines are curved and layered, creating a sense of depth and movement, reminiscent of light trails or data flow.

Demo

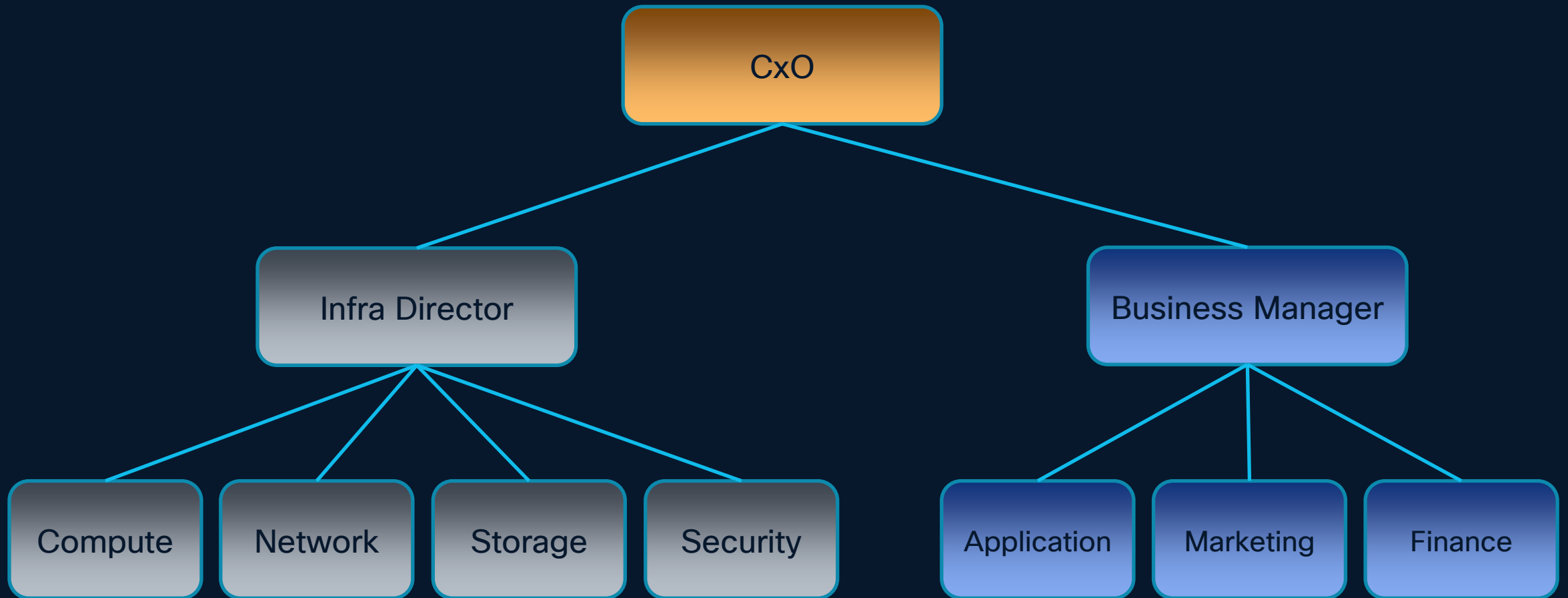
Demo – Revisit Deploy ACI Bridge Domain



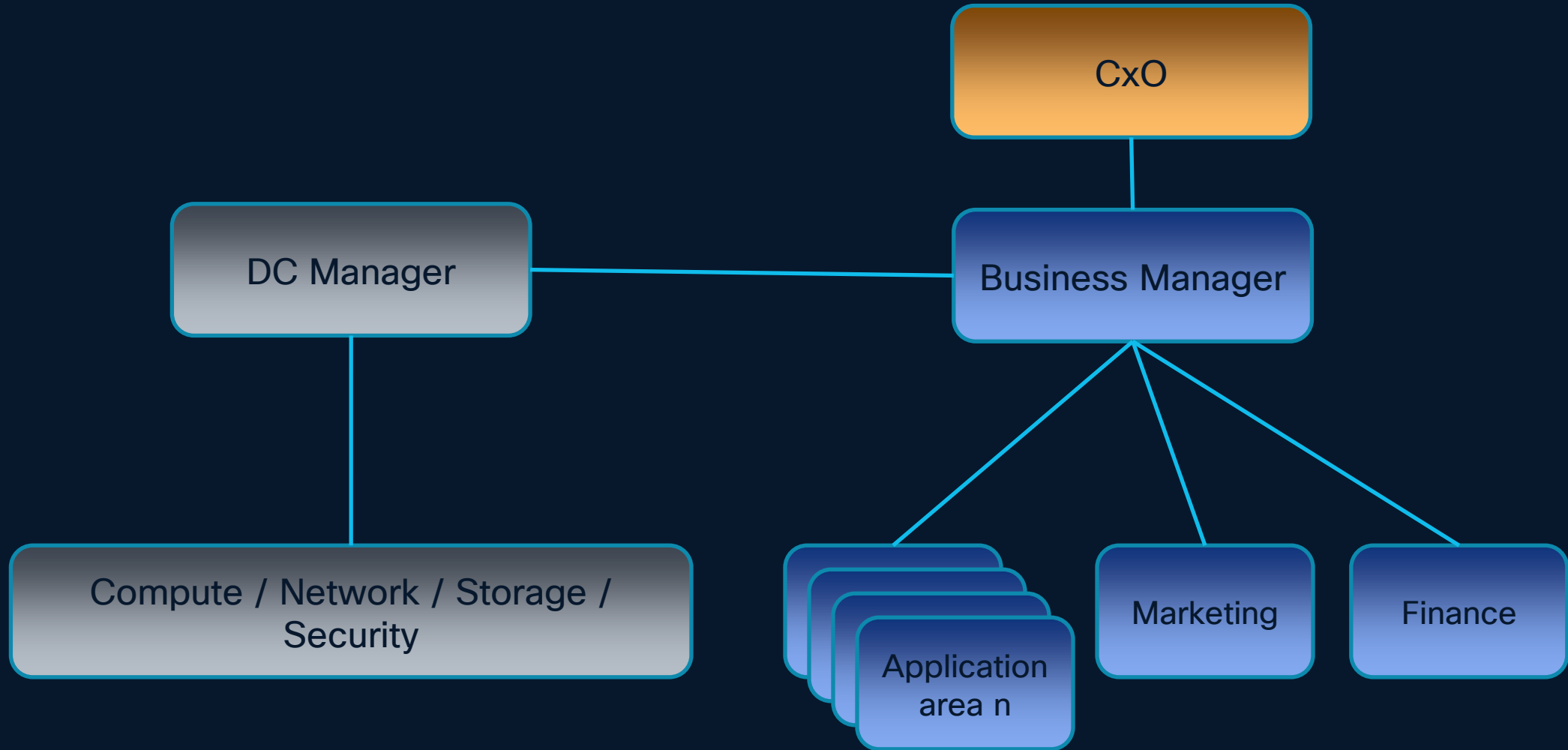
Organizational Challenges

*How many of you see the
organization as a challenge to
achieve full automation?*

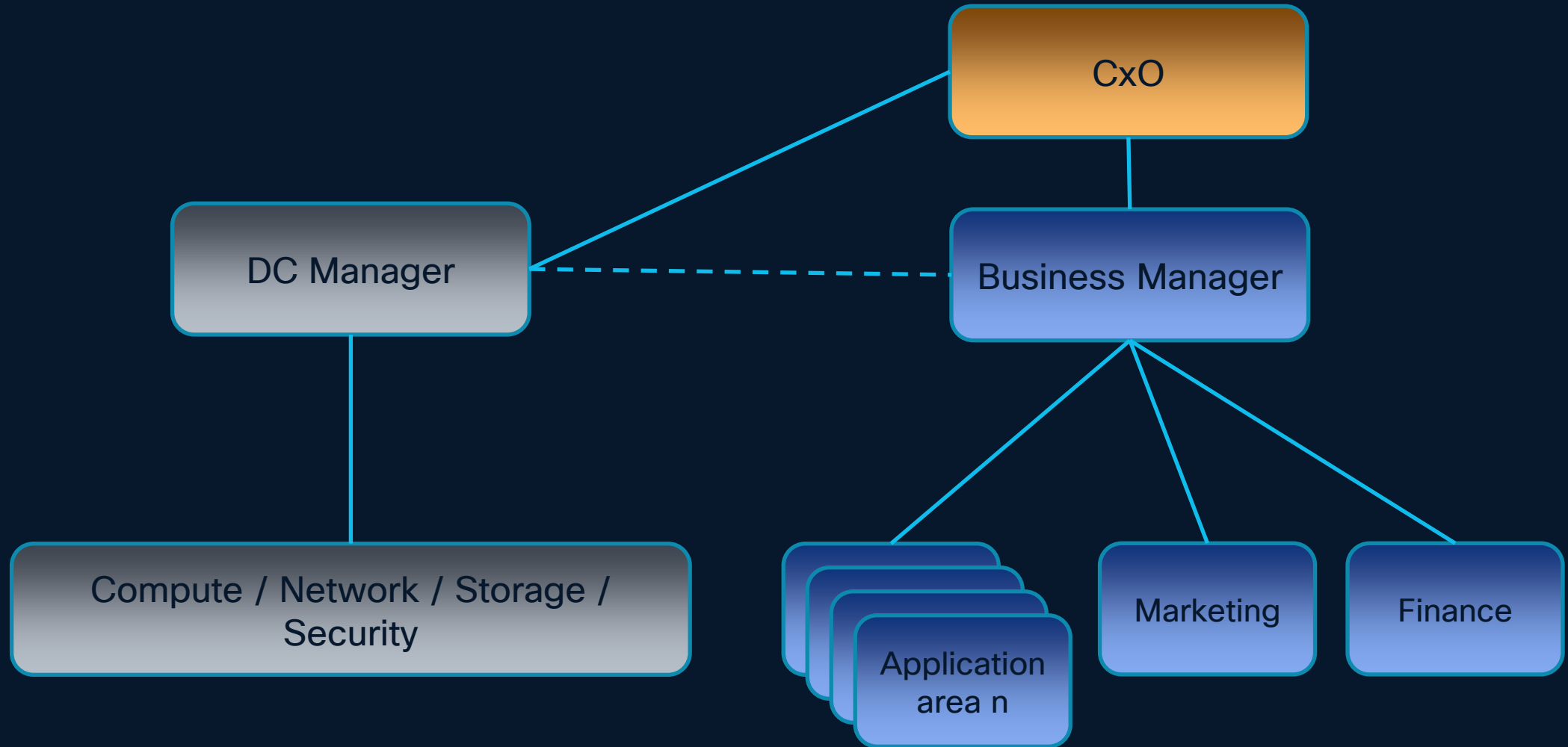
Classical Structure



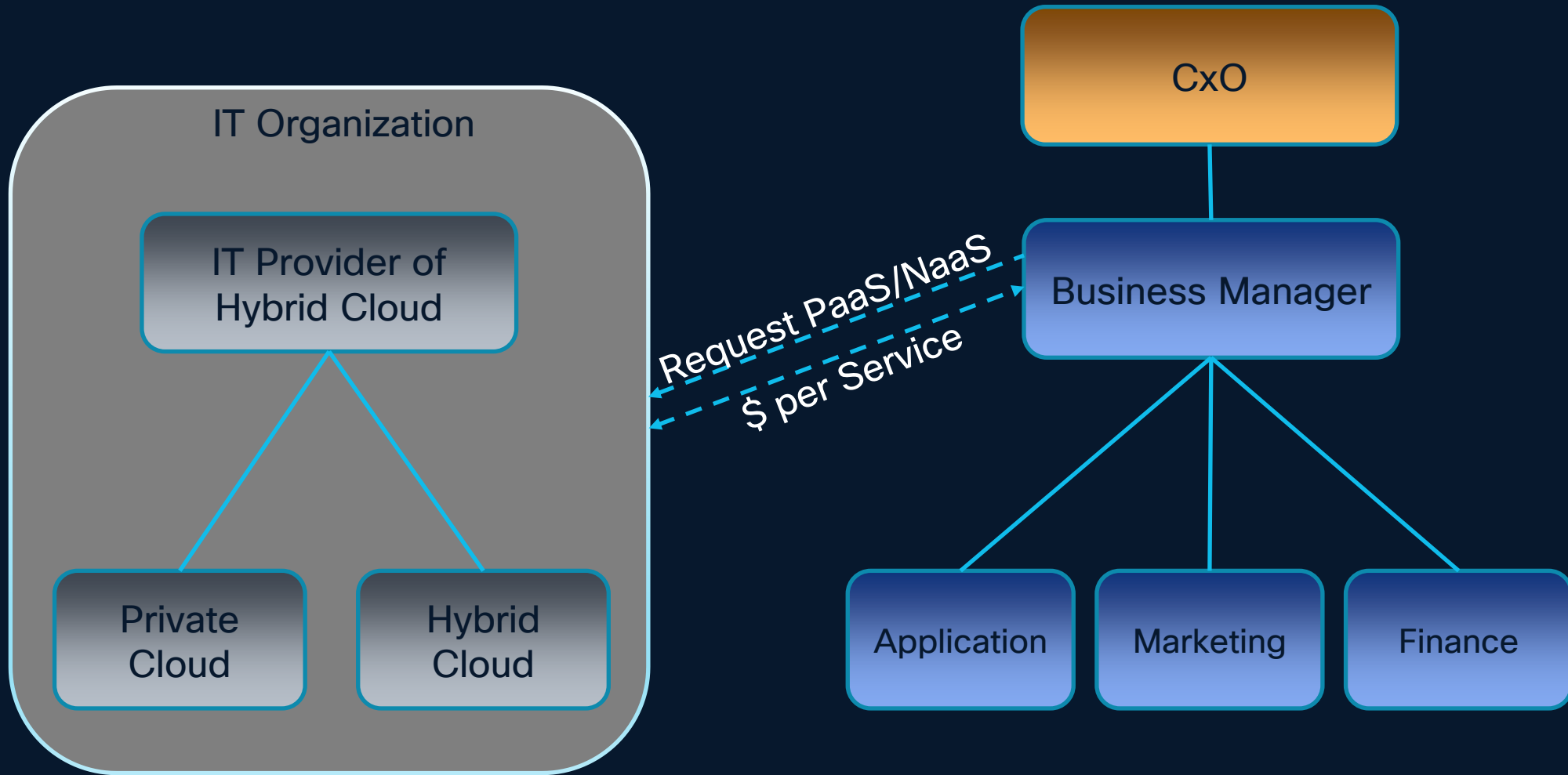
Classical Approach but Infra Aligned



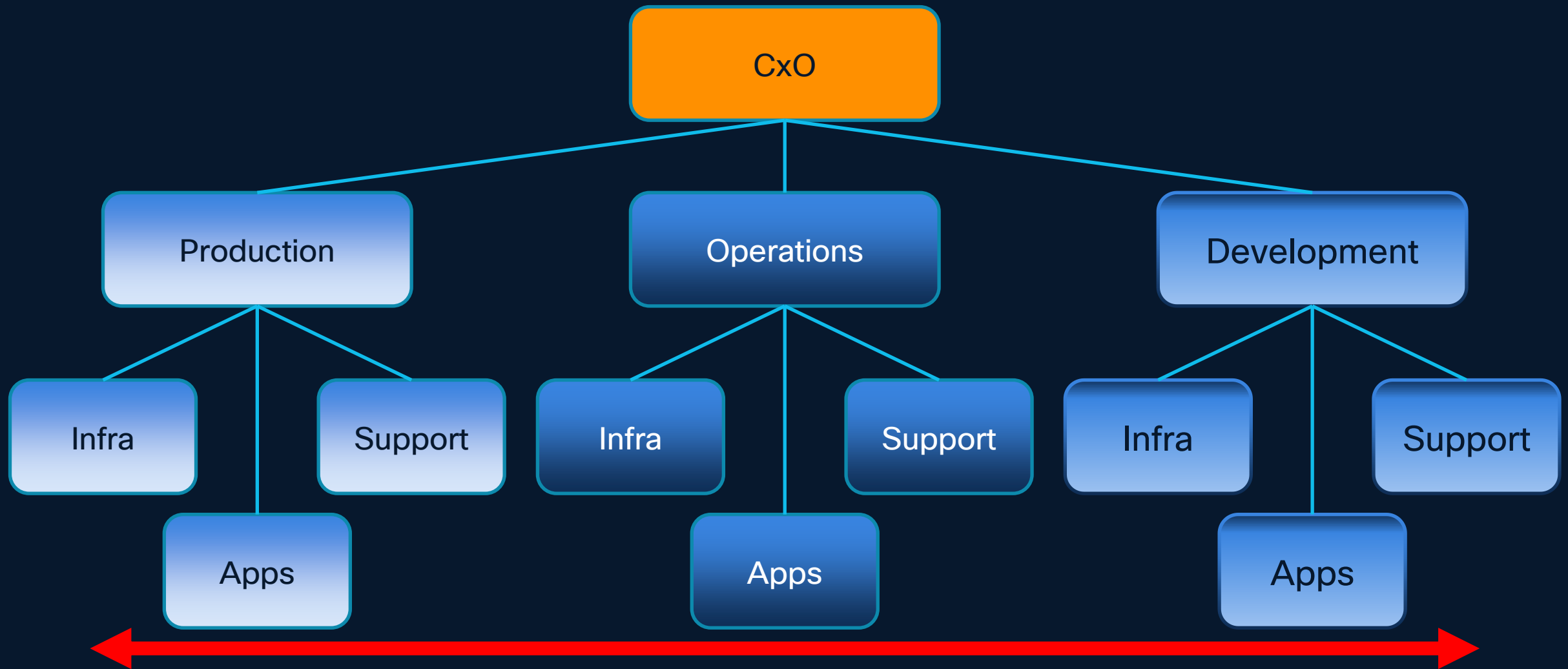
Classical Structure with a link to C-Level



IT as a Service Provider inside the Enterprise



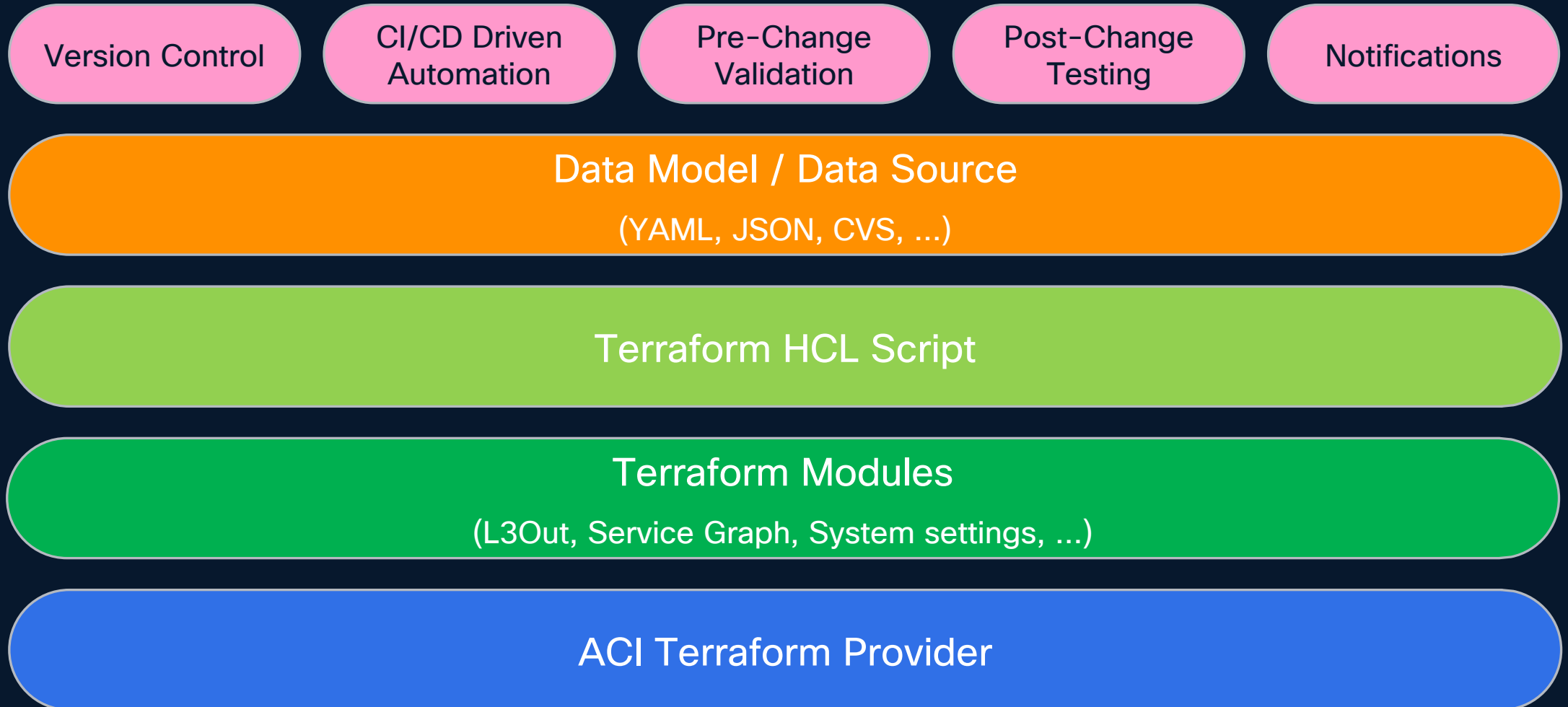
Squad based approach



Lack of Coordination of Knowledge and Technology Choices

ACI as Code + Demo

Solution Overview – ACI as Code



Separate Data from Code

In order to ease maintenance and enable separation of duties we separate data (variable definition) from logic (infrastructure declaration), where one can be updated independently from the other

```
apic:
  tenants:
    - name: CiscoLive
  vrfs:
    - name: CiscoLive
  bridge_domains:
    - name: vlan-100
      vrf: CiscoLive
  application_profiles:
    - name: dev
      endpoint_groups:
        - name: vlan-100
          bridge_domain: vlan-100
          physical_domains: ["L2"]
```

apic.yaml

```
module "aci" {
  source = "netascode/nac-aci/aci"
  version = "0.9.3"

  yaml_directories = ["data"]

  manage_access_policies = true
  manage_fabric_policies = true
  manage_pod_policies = true
  manage_node_policies = true
  manage_interface_policies = true
  manage_tenants = true
}
```

main.tf

ACI Configuration / YAML Layout

- As different teams might be responsible for different parts of the infrastructure, it is important to allow flexibility
- The configuration can be divided into multiple YAML files
- Network as Code does not dictate a specific file structure, but allows for full flexibility to divide the configuration as needed

```
$ tree -L 2
```

```
.  
├── data  
│   ├── apic.yaml  
│   ├── access_policies.yaml  
│   ├── fabric_policies.yaml  
│   ├── node_policies.yaml  
│   ├── pod_policies.yaml  
│   ├── node_1001.yaml  
│   ├── node_101.yaml  
│   ├── node_102.yaml  
│   ├── tenant_PROD.yaml  
│   └── defaults.yaml  
└── main.tf
```

ACI Pre-Change Validation - Syntax

- Native Terraform variable validation rules have limitations with complex and/or nested structures
- Tools like [Yamale](#) can be used to define the schema and validate YAML files against it
- The schema specifies the expected structure, value types, and constraints

```
apic: include('apic', required=False)
---
apic:
  tenants: list(include('tenant'),
                 required=False)
tenant:
  name: regex('^[a-zA-Z0-9_.-]{1,64}$')
  vrfs: list(include('ten_vrf'), required=False)
ten_vrf:
  name: regex('^[a-zA-Z0-9_.-]{1,64}$')
  alias: regex('^[a-zA-Z0-9_.-]{1,64}$',
               required=False)
  data_plane_learning: bool(required=False)
  enforcement_direction: bool(required=False)
  contracts: include('ten_vrf_contracts',
                    required=False)
```

ACI Pre-Change Validation -

Semantic validation verifies specific data model related constraints like referential integrity, etc.

Examples are:

- Check uniqueness of key values (eg. Node IDs, Object Names, etc.)

- Check references/relationships between objects (ex. Interface Policy Group referencing a CDP Policy)

```
Rule 101: Verify unique keys ['apic.node_policies.nodes.id - 102']
```

```
Rule 201: Verify references ['apic.node_policies.nodes.update_group - GROUP1']
```

```
Rule 205: Verify Access Spine Interface Policy Group references
```

```
['apic.interface_policies.nodes.interfaces.policy_group - SERVER1']
```


ACI Post-Change Testing

Post-Change testing provides quality assurance for the performed change, ensuring that it was performed successfully without causing an outage.

For ACI are the three test categories performed the following way

Configuration Tests

- Can be verified using "terraform plan"
- Can be verified using post-change config tests

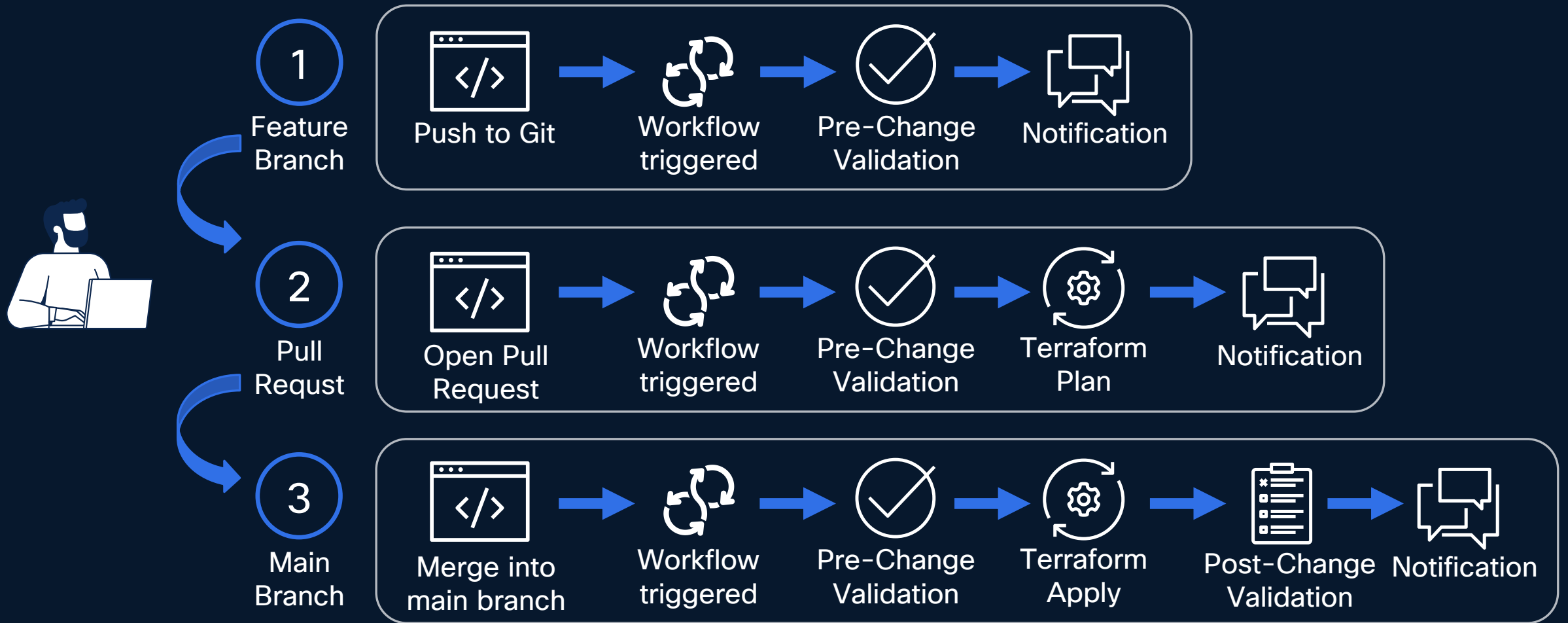
Health Tests

- Pulls fault and health scores from APIC
- Compares the fault/health scores against either a static threshold or against value before change

Operational Tests

- Pulls the operational state of BGP sessions, etc.

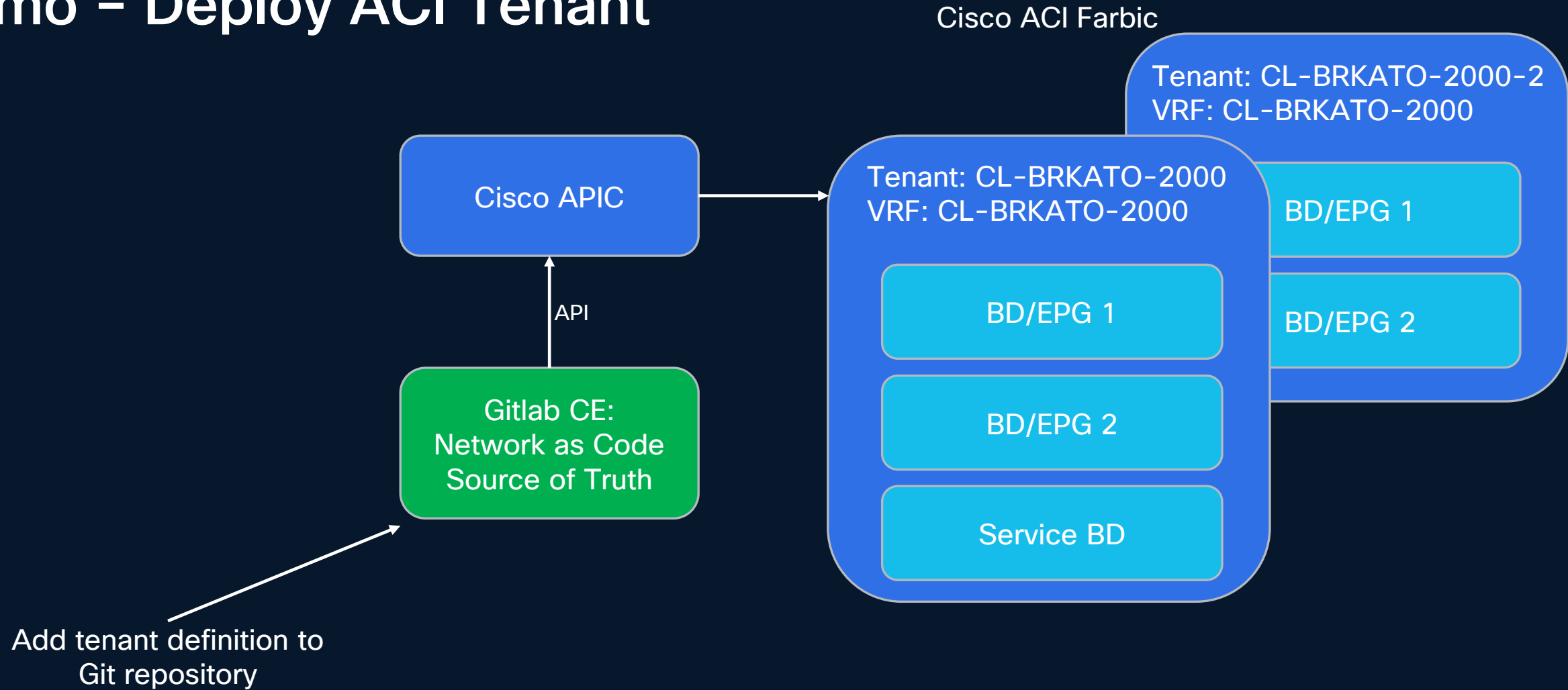
Recap: ACI as Code Workflow



The background of the slide is a dark blue field filled with numerous bright, glowing blue and orange light streaks. These streaks are curved and flow across the frame, creating a sense of motion and energy. A solid dark blue rectangular box is positioned on the left side of the slide, serving as a backdrop for the title text.

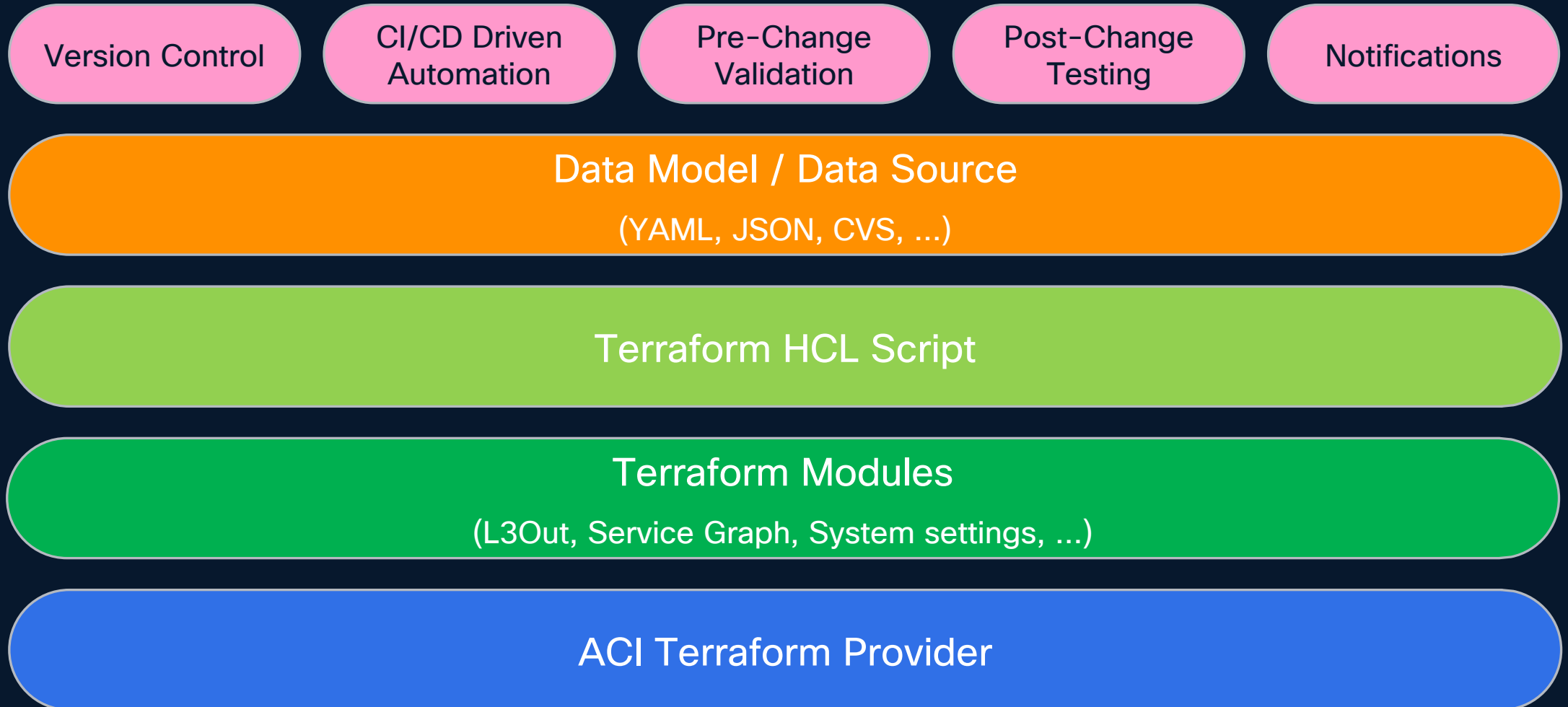
Demo

Demo – Deploy ACI Tenant

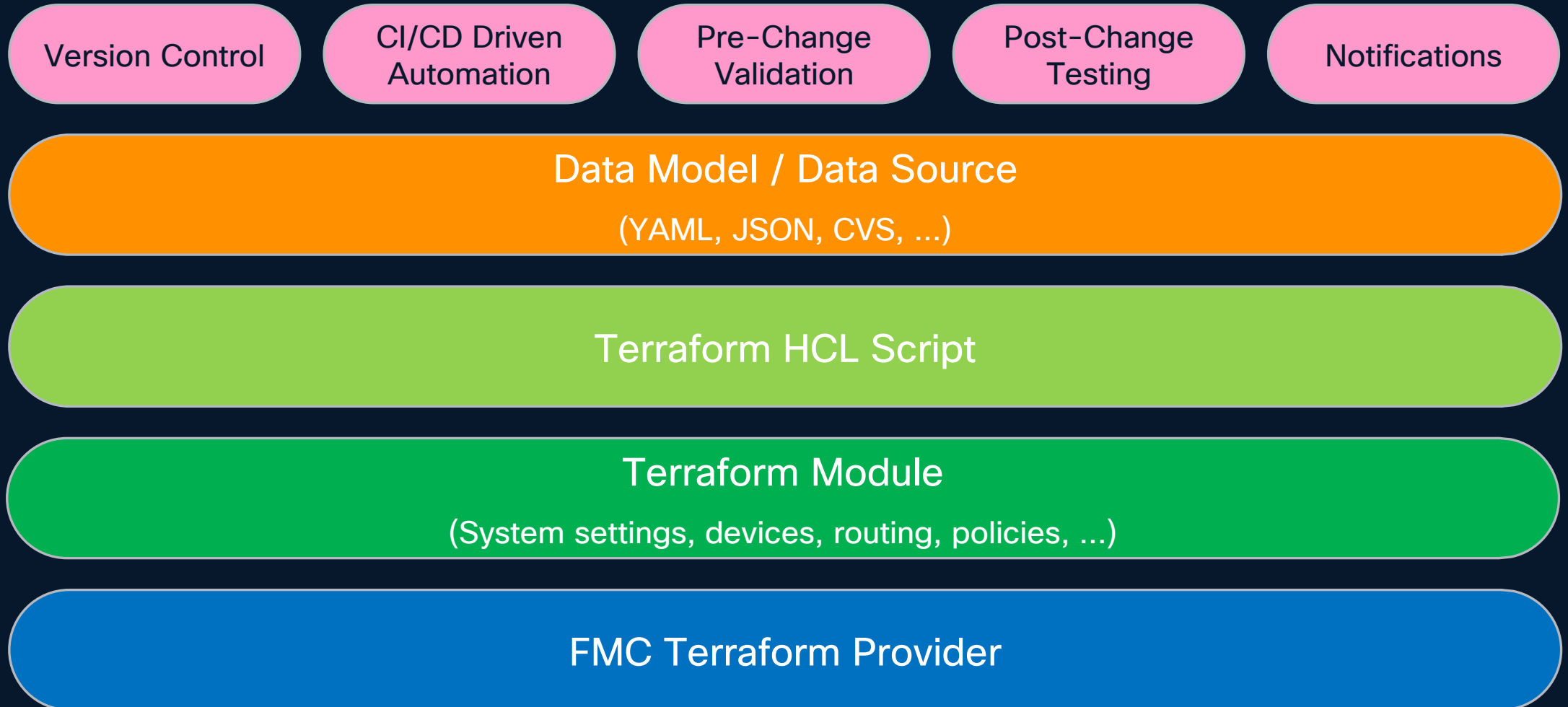


Firepower/FMC as Code + Demo

Recap: Solution Overview - ACI as Code



Solution Overview – Firepower/FMC as Code



Standardization drives simplicity

All Network as Code solutions are build using the same principles, tools, etc. to simplify adoption across architectures.

FMC/Firepower as Code uses the same separation of data/code as seen for ACI, as well as the same tools for pre- and post-deployment validation

```
fmc:
  domains:
    - name: CiscoLive
  policies:
    - access_policies: DC_SECURITY
      default_action: BLOCK
      categories:
        - name: category1
      access_rules:
        - name: allow_http
          action: allow
          category: category1
          source_zones:
            - outside
          destination_zones:
            - inside
  ...
```

fmc.yaml

```
module "fmc" {
  source = "netascode/nac-fmc/fmc"
  version = "0.0.2"

  yaml_directories = ["data"]

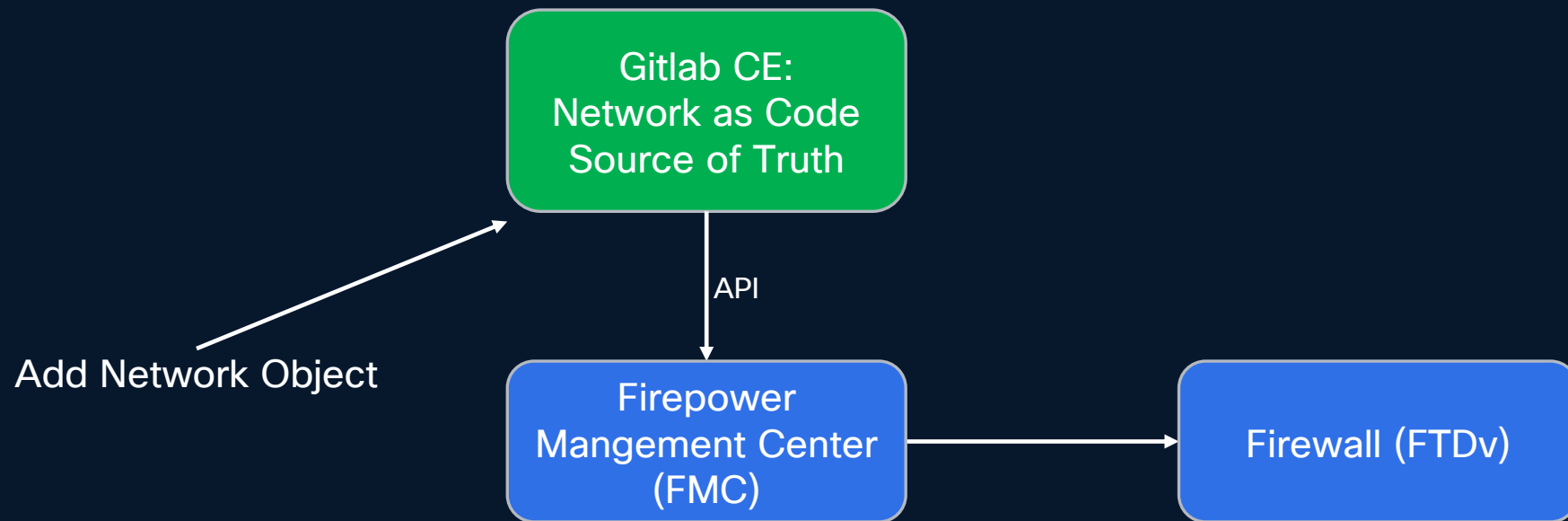
  manage_deployment = true
}
```

main.tf

The background of the slide is a dark blue field filled with dynamic, flowing light streaks. These streaks are primarily in shades of bright blue and cyan, with some warmer orange and pinkish tones visible in the upper right quadrant. The lines are curved and layered, creating a sense of depth and movement, reminiscent of light trails or data flow.

Demo

Demo – Deploy Network Object



Standardize IT offerings

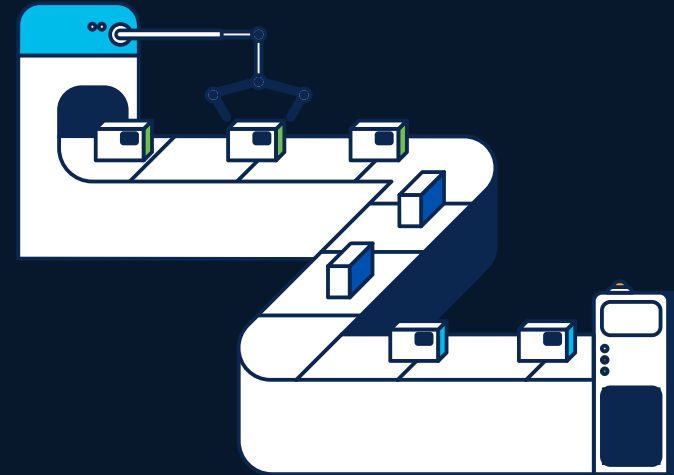
Why Standardize of IT offerings

"A La Carte" IT Service



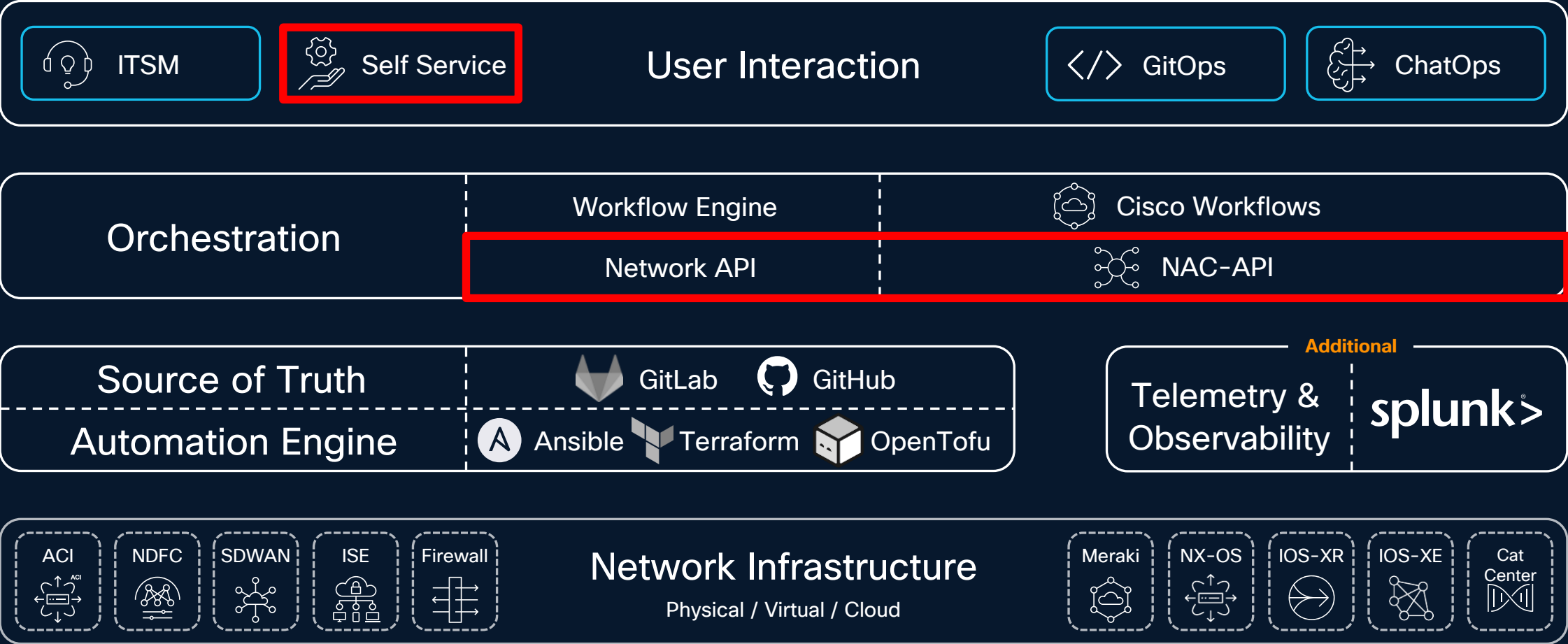
- Enables end-users to select/modify the service attributes as they see fit
- Difficult to expose through a service catalogue
- Costly to operate at scale, as each service instance will be unique

Standardized IT Service



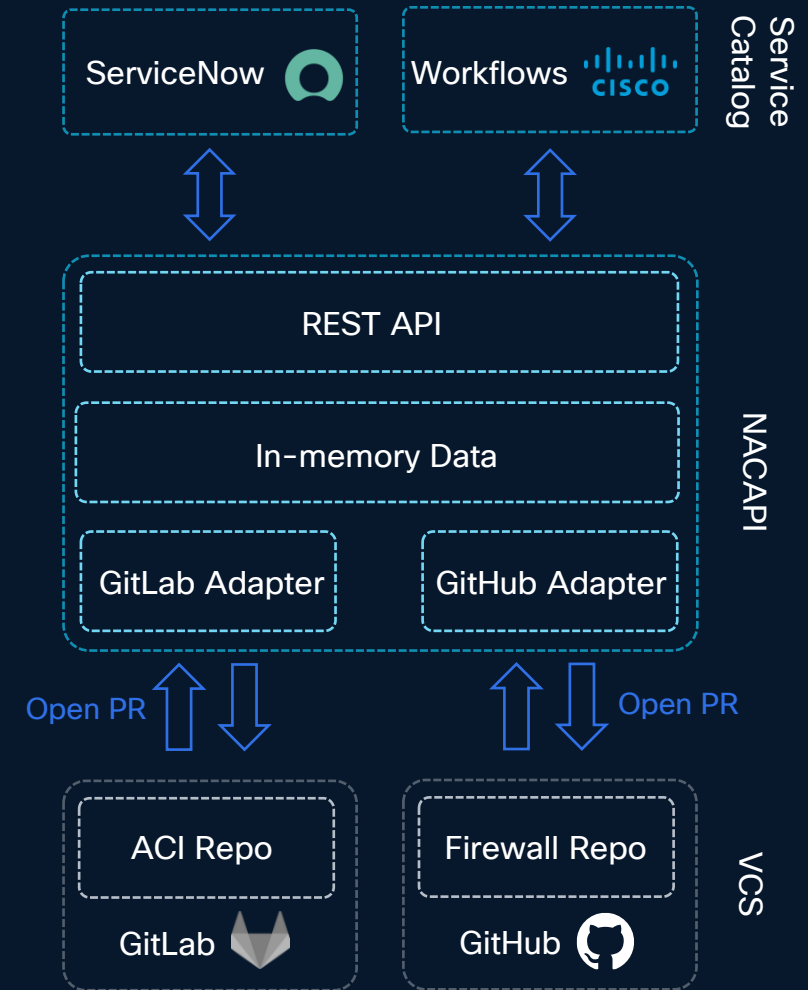
- Enables end-users to select/modify a limited set of service attributes
- Easy to expose through a service catalogue
- Easier to operate at scale as large number of service instances will be identical

Recap: Network as Code Architecture

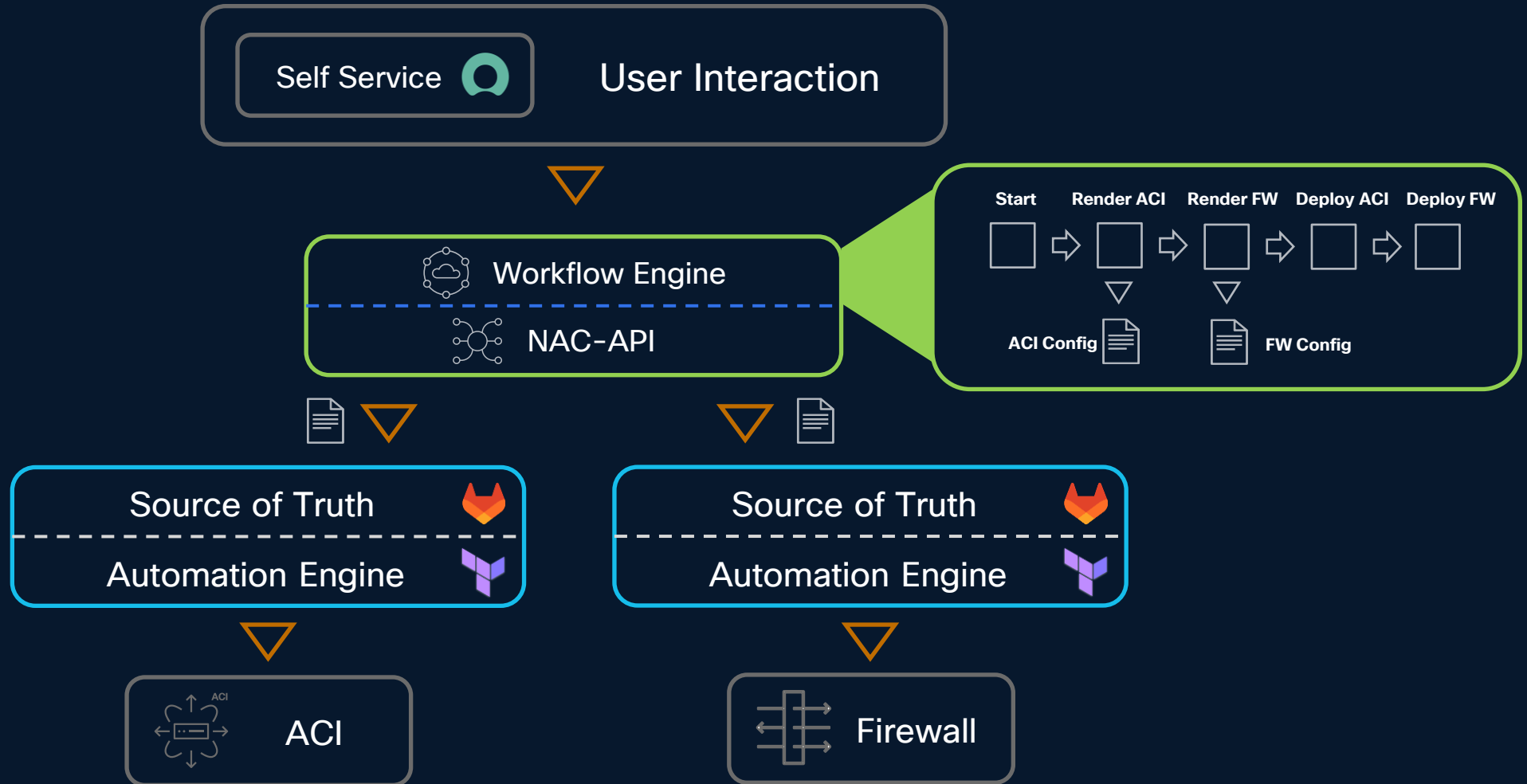


Network as Code API (NAC-API) Architecture

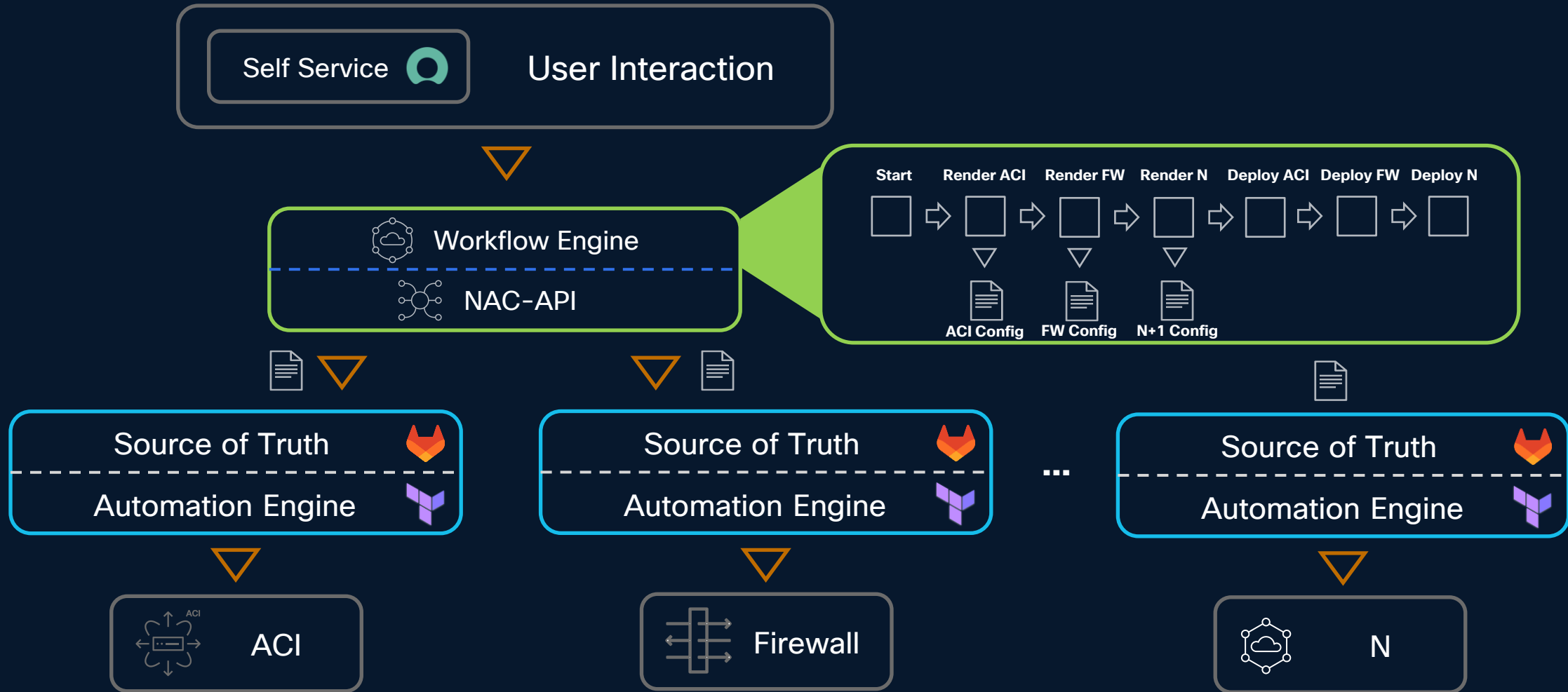
- Containerized component exposing
 - A REST API for Create, Read, Update, and Delete (CRUD) operations
 - GraphQL API to query Network as Code state
- REST API can be consumed by service catalogs / portal (i.e. ServiceNow, Cisco Workflows, Homegrown Portal, etc.), workflow engines, chatbots or GUIs
- REST CRUD operation mimics the workflow done by operators
- GraphQL can be used by tools like the AI Assistant to query the intended configuration
- Adapters to connect to different VCS systems (GitLab, GitHub, Bitbucket, Azure DevOps)



Network as Code – Cross Domain Automation



Network as Code – Cross Domain Automation

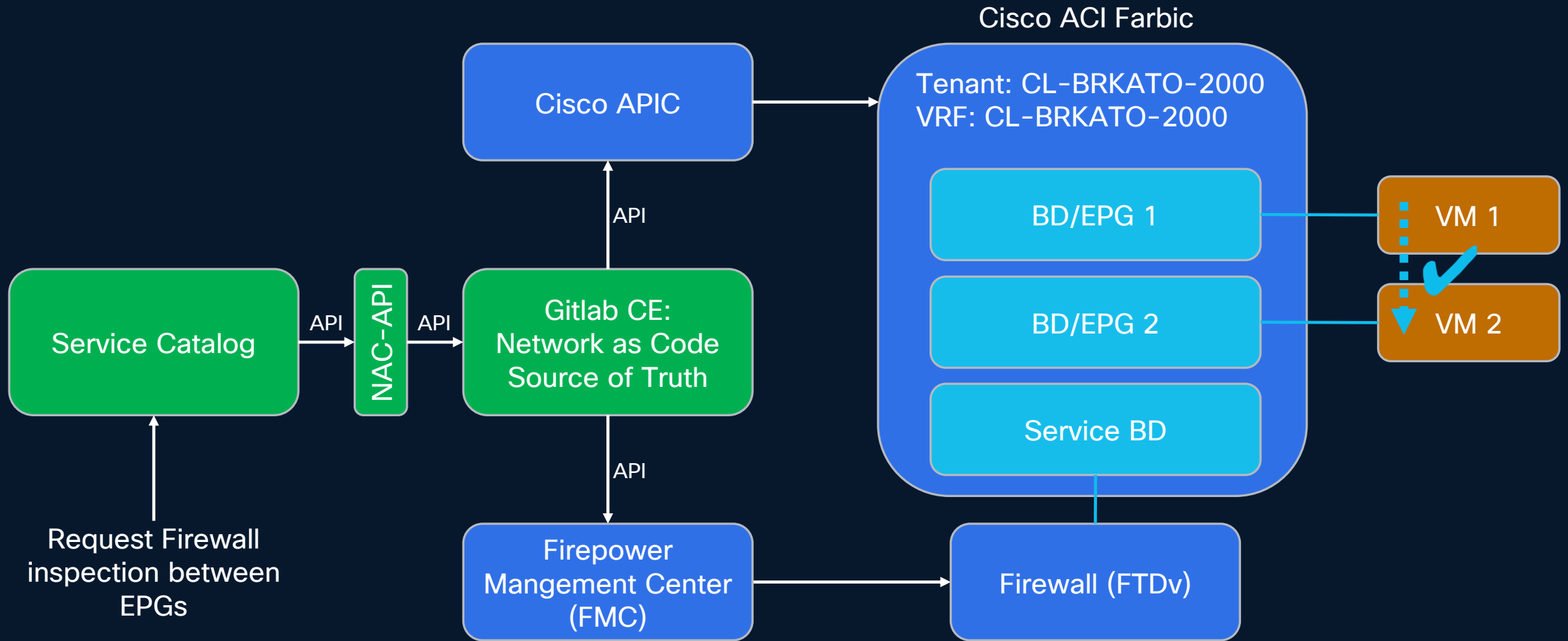


N = Other Cisco Architecture(s)

The background of the slide is a dark blue field filled with dynamic, flowing light streaks. These streaks are primarily in shades of bright blue and cyan, with some warmer orange and pinkish tones visible in the upper right quadrant. The lines are curved and layered, creating a sense of depth and movement, reminiscent of light trails or data flow.

Demo

Demo – Insert Firewall inspection between EPGs



Conclusion

Common Pitfalls

Tribal Knowledge

Processes are
old and not
adapted

Organisation is
not ready

Silos

No roadmap to
follow

Overcomplicating

Measurable KPIs



Complete your session surveys



Complete your surveys in the Cisco Events App.



Complete a minimum of 4 session surveys and the overall event survey to receive a unique Cisco Live t-shirt.
(from 11:30 on Thursday, while supplies last)

Continue your education



Visit the Cisco Showcase for related demos



Book your one-on-one Meet the Engineer meeting

Visit the Technical Solutions Clinics to discuss your technical questions



Attend the interactive education with DevNet, Capture the Flag, and Walk-in Labs



Visit the On-Demand Library for more sessions at CiscoLive.com/On-Demand

Thank you

CISCO Live !

